# Cloud Databases: A Systematic Meta-Survey of Current Challenges, Emerging Paradigms, and Future Research Directions

Abdulhakeem Ibrahim[a,*]

[a]School of Computer Science and Informatics, De Montfort University, Leicester, United Kingdom

ARTICLE INFO

*Keywords*:
cloud databases
systematic meta-survey
cloud-native databases
database-as-a-service
data lakehouse
AI4DB
survey of surveys

ABSTRACT

Cloud databases have become the backbone of modern data-driven applications, enabling organizations to store, process, and analyze data at unprecedented scale and elasticity. Over recent years (2016–2025), 24 survey papers have examined various facets of cloud database technology—spanning cloud-native architectures, hybrid transactional and/or analytical processing (HTAP), NoSQL/NewSQL systems, vector and graph databases, AI-for-database optimization (AI4DB), data lakehouses, time-series databases, cloud database migration, and edge–cloud data management. However, the challenges, open problems, and research directions identified in these surveys remain fragmented across disparate communities and publication venues. This paper presents the first *systematic meta-survey* of cloud database research: a survey of surveys that synthesizes findings from carefully selected survey papers using a rigorous methodology grounded in the Kitchenham–Charters systematic literature review protocol. The scattered landscape of cloud database challenges is organized into a structured, dual-theme taxonomy: (1) *eighteen cross-cutting challenges* that transcend individual system categories, including scalability, consistency trade-offs, security, cost optimization, AI–database convergence, and sustainability; and (2) *five lifecycle-phase challenges* spanning design, deployment, operation, optimization, and evolution of cloud database systems. The convergence of cloud databases with large language models (LLMs) is further examined as an illustrative case study, and a consolidated set of open research directions is distilled from the cross-survey synthesis. By providing a consolidated, thematically organized view of the field, this meta-survey serves as both a comprehensive reference for researchers entering the cloud database space and a strategic guide for practitioners navigating the rapidly evolving landscape of cloud data management.

## 1. Introduction

The cloud database market has experienced explosive growth over the past decade, with global revenues projected to exceed $200 billion by 2028 [1]. This growth reflects a fundamental shift in how organizations manage data: from on-premises, monolithic database servers to elastic, globally distributed, cloud-native data platforms that can scale storage and compute independently on demand [2, 3]. Major cloud providers—Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and Alibaba Cloud—now offer dozens of specialized database services, from relational engines like Amazon Aurora [2] and Azure SQL [4] to purpose-built systems for documents, graphs, time series, key-value pairs, and, most recently, vector embeddings for AI workloads [5].

This rapid evolution has attracted significant research attention. A search across major digital libraries reveals that **24 survey papers** meeting the inclusion criteria were published between 2016 and 2025, each examining a particular facet of cloud database technology. These surveys span a wide spectrum of topics: cloud-native database architectures [6], hybrid transactional/analytical processing (HTAP) [7], graph databases [8], vector database management systems [5], AI-driven database optimization (AI4DB) [9], data lakehouses [10], NoSQL and NewSQL systems [11, 12], edge and fog database systems [13], and many more.

Despite this wealth of individual surveys, the field lacks a *meta-level synthesis*—a comprehensive, systematic effort to consolidate the challenges, open problems, and research directions scattered across these surveys into a unified framework. Each survey naturally focuses on its own domain, leading to a fragmented understanding of the cloud database landscape as a whole. Cross-cutting challenges such as cost optimization, security, consistency trade-offs, and AI–database convergence are discussed in many surveys but from different angles, without a common organizing framework.

*Corresponding author.
hakeem.ibrahim@dmu.ac.uk (A. Ibrahim)
ORCID(s): 0000-0002-1104-5314 (A. Ibrahim)

**Table 1**
Research questions guiding the meta-survey.

| ID | Research Question |
|---|---|
| RQ1 | What is the landscape of cloud database surveys published between 2016 and 2025, in terms of topic coverage, methodology, and publication venue? |
| RQ2 | What are the major cross-cutting challenges identified across cloud database surveys, and how do they relate to one another? |
| RQ3 | How do cloud database challenges map to the lifecycle phases of designing, deploying, operating, optimizing, and evolving database systems? |
| RQ4 | What is the current state and future trajectory of the convergence between cloud databases and AI/ML technologies, particularly large language models? |
| RQ5 | What are the most pressing open research directions, and which of them recur most widely across the surveyed literature? |

## 1.1. Contributions and novelty

This paper addresses the gap by presenting the **first systematic meta-survey of cloud database research**—a survey of surveys. A rigorous systematic literature review (SLR) protocol based on the Kitchenham–Charters guidelines [14] is applied to identify, evaluate, and synthesize cloud database surveys. The contributions of this work are as follows:

**C1 Systematic identification and classification.** Eight digital libraries are systematically searched, yielding 24 survey papers on cloud databases published between 2016 and 2025, classified by topic, scope, methodology, and venue.

**C2 Dual-theme challenge taxonomy.** The scattered challenges are organized into a structured taxonomy with two complementary themes:

- *Theme 1: Eighteen cross-cutting challenges* that recur across multiple survey domains, including scalability, consistency, security, cost modeling, AI–database convergence, and sustainability.
- *Theme 2: Five lifecycle-phase challenges* that map to the stages of designing, deploying, operating, optimizing, and evolving cloud database systems.

**C3 LLM–database convergence analysis.** The rapidly emerging intersection of large language models (LLMs) and cloud databases is examined as an illustrative case study, covering text-to-SQL, vector databases for retrieval-augmented generation (RAG), and LLM-based database administration.

**C4 Research agenda.** A consolidated, cross-cutting research agenda is distilled from the synthesis, grouping the open problems that recur most widely across the surveyed literature.

**C5 Knowledge engineering perspective.** Cloud database challenges are explicitly connected to data and knowledge engineering concerns—including knowledge graphs, ontologies, metadata management, and schema evolution—making this survey particularly relevant to the *Data & Knowledge Engineering* community.

## 1.2. Research questions

This meta-survey is guided by five research questions, summarized in Table 1 and elaborated in Section 4.

## 1.3. Paper organization

The remainder of this paper is organized as shown in Figure 1. Section 2 motivates the importance of cloud databases from five complementary perspectives. Section 3 establishes key definitions and distinctions—from cloud-hosted to cloud-native databases, architectural paradigms, data models, and the data platform continuum. Section 4 details the systematic review methodology, including search strategy, selection criteria, quality assessment, and results.

Section 5 presents the core contribution: a comprehensive discussion of challenges and research directions organized by cross-cutting themes and lifecycle phases. Section 6 examines the convergence of cloud databases and LLMs. Section 7 concludes. Appendix A provides the complete catalog of surveyed papers.


Figure 1: The organization of this meta-survey paper.

## 2. Why Cloud Databases Matter—A Multi-Perspective Motivation

Understanding the significance of cloud databases requires examining the topic from multiple viewpoints. In this section, the need for a meta-survey is motivated by discussing five complementary perspectives (Figure 2), each revealing why the scattered state of survey knowledge is problematic and why a unified synthesis is timely.

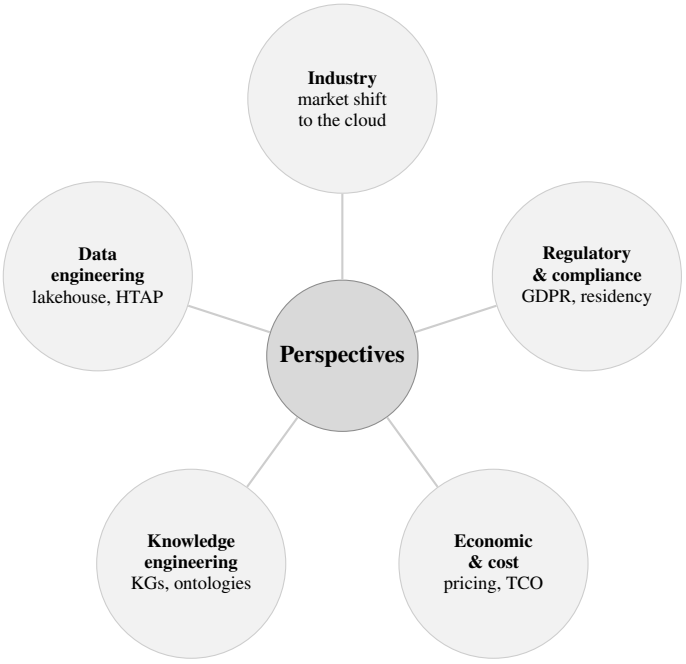


Figure 2: The five complementary perspectives motivating a systematic meta-survey of cloud databases.

## 2.1. Industry perspective

The global database management system (DBMS) market reached $101 billion in 2023, with cloud deployments accounting for over 65% of total revenue—up from approximately 40% in 2019 [1]. Gartner projects that by 2026, 75% of all databases will be deployed on cloud platforms [1]. This shift is driven by digital transformation initiatives across industries, from financial services adopting cloud databases for real-time fraud detection to healthcare organizations leveraging cloud data platforms for genomics and precision medicine [15].

The vendor landscape has also diversified significantly. Beyond the hyperscale cloud providers (AWS, Azure, GCP), specialized cloud database vendors such as Snowflake [3], Databricks [16], CockroachDB [17], PlanetScale, and Neon have gained substantial market share. China's cloud database ecosystem—including Alibaba PolarDB [18], TiDB [19], and OceanBase [20]—has matured rapidly. This fragmentation makes it difficult for practitioners to navigate the landscape without a systematic overview.

## 2.2. Data engineering perspective

Modern data architectures have evolved from simple transactional databases to complex, multi-layered data ecosystems. The emergence of the *data lakehouse* paradigm [16] exemplifies this evolution, combining the flexibility of data lakes with the governance and performance of data warehouses. Systems like Databricks Delta Lake, Apache Iceberg, and Apache Hudi have created a new "open table format" layer that bridges batch and streaming analytics [10].

Simultaneously, the rise of *HTAP* (Hybrid Transactional/Analytical Processing) systems [7] challenges the traditional separation between OLTP and OLAP workloads. Systems such as TiDB [19], SAP HANA [21], and Google AlloyDB promise unified engines that handle both transaction processing and analytical queries. Understanding the trade-offs, maturity, and open challenges of these paradigms requires synthesizing insights across multiple survey papers.

## 2.3. Knowledge engineering perspective

Cloud databases play a crucial role in knowledge engineering, making this topic directly relevant to the *Data & Knowledge Engineering* (DKE) community. Several connections are worth highlighting:

- **Knowledge graphs in the cloud.** Large-scale knowledge graphs (KGs) such as Google Knowledge Graph, Wikidata, and enterprise KGs are increasingly stored and queried using cloud-hosted graph databases (e.g., Amazon Neptune, Azure Cosmos DB with Gremlin API, Neo4j Aura) [8, 22, 23]. The scalability, consistency, and query optimization challenges of cloud graph databases directly impact KG applications.

- **Ontology-mediated data access.** Cloud environments with heterogeneous data sources benefit from ontology-based data integration (OBDI) techniques [24], where ontologies provide a unified conceptual layer over distributed cloud databases.
- **Metadata and schema management.** Cloud data catalogs and metadata management systems (e.g., AWS Glue Data Catalog, Apache Atlas) implement knowledge engineering principles for organizing, discovering, and governing data assets [25].
- **AI–knowledge engineering convergence.** The integration of machine learning with database systems (AI4DB) [9] draws on knowledge engineering concepts such as learned models for query optimization, workload characterization, and automated tuning.

### 2.4. Economic and cost perspective

Cloud databases introduce novel economic dynamics compared to on-premises deployments. The shift from capital expenditure (CapEx) to operational expenditure (OpEx) models, combined with pay-per-use pricing and complex multi-dimensional billing (compute, storage, I/O, network egress, backups), creates significant challenges in cost prediction and optimization [26, 27].

Research has shown that cloud database costs can vary by orders of magnitude depending on configuration choices, query patterns, and pricing model selection [26]. The total cost of ownership (TCO) analysis becomes particularly complex for multi-cloud or hybrid cloud deployments. Several surveys touch on cost-related challenges [6, 28], but a consolidated analysis is lacking.

### 2.5. Regulatory and compliance perspective

Data sovereignty regulations such as the European Union's General Data Protection Regulation (GDPR) [29], China's Data Security Law, and sector-specific requirements (e.g., HIPAA for healthcare, PCI-DSS for payment data) impose strict constraints on how and where cloud databases store and process data [30].

Multi-region deployment, data residency requirements, cross-border data transfer restrictions, and audit trail obligations all impact cloud database architecture and operations. The compliance landscape interacts with technical challenges such as geo-distributed consistency, encryption, access control, and data lineage—topics discussed in various surveys but rarely addressed in an integrated manner.

## 3. From Cloud Databases to Cloud-Native Data Platforms

Before analyzing the surveyed literature, key concepts, definitions, and distinctions are established that form the vocabulary of cloud database research.

### 3.1. Cloud-hosted vs. cloud-native vs. cloud-born databases

The term "cloud database" encompasses a spectrum of architectures with fundamentally different design philosophies. Three categories are distinguished, compared in Table 2:

**Cloud-hosted databases** are traditional database engines (e.g., PostgreSQL, MySQL, Oracle) deployed on cloud virtual machines. They benefit from cloud infrastructure (networking, storage volumes, automated backups) but do not exploit cloud-native primitives such as elastic compute pools or disaggregated storage [6].

**Cloud-native databases** are existing or new engines that have been substantially re-architected to leverage cloud services. Amazon Aurora [2] exemplifies this approach: it re-implements the storage layer of MySQL/PostgreSQL to use a distributed, log-structured storage service while keeping the SQL engine largely intact. Microsoft's Socrates [4] takes SQL Server and disaggregates its components across cloud microservices.

**Cloud-born databases** are designed entirely from the ground up for cloud deployment, with no legacy on-premises heritage. Snowflake [3] pioneered this model for data warehousing, while CockroachDB [17] and Neon represent cloud-born approaches for transactional and PostgreSQL-compatible workloads, respectively.

### 3.2. Database-as-a-Service (DBaaS) spectrum

Database-as-a-Service (DBaaS) represents the fully managed end of the cloud database spectrum, where the cloud provider handles provisioning, patching, scaling, backup, and high availability [31, 32, 33, 34, 35]. The DBaaS model has evolved along several dimensions:

**Table 2**
Comparison of cloud database deployment models.

| Characteristic | Cloud-Hosted | Cloud-Native | Cloud-Born |
|---|---|---|---|
| **Definition** | Traditional DBMS deployed on cloud VMs | DBMS redesigned to exploit cloud infrastructure | DBMS designed from scratch for the cloud |
| **Architecture** | Monolithic, tightly coupled | Disaggregated compute/storage | Fully disaggregated, microservice-based |
| **Elasticity** | Manual scaling (vertical) | Automatic scaling (horizontal) | Instant, fine-grained scaling |
| **Examples** | MySQL on EC2, Oracle on Azure VM | Amazon Aurora [2], Azure SQL [4] | Snowflake [3], Neon, CockroachDB [17] |
| **State sharing** | Shared-nothing (typically) | Shared-storage | Disaggregated, shared cloud services |
| **Multi-tenancy** | VM-level isolation | Engine-level isolation | Native multi-tenant design |

- **Managed databases**: Provider-managed instances of open-source or commercial engines (e.g., Amazon RDS, Azure Database for PostgreSQL).
- **Serverless databases**: Fully elastic databases that scale to zero when idle and automatically provision resources based on demand (e.g., Amazon Aurora Serverless, Azure Cosmos DB serverless, Neon) [28].
- **Autonomous databases**: Self-driving databases that use ML to automate tuning, indexing, partitioning, and anomaly detection (e.g., Oracle Autonomous Database) [36].

## 3.3. Architectural paradigms

Cloud database architectures have evolved through three major paradigms, illustrated in Figure 3:

**Shared-nothing** architectures partition data across independent nodes, each with its own CPU, memory, and disk. Traditional distributed databases (e.g., early Spanner [37], Cassandra) follow this model. While horizontally scalable, resharding and rebalancing are expensive.

**Shared-disk (cloud storage)** architectures decouple compute from storage, allowing compute nodes to share a common cloud storage layer. Amazon Aurora [2] and Snowflake [3] exemplify this paradigm, enabling independent scaling of compute and storage.

**Fully disaggregated** architectures further decompose the database engine into independently scalable microservices—query processing, transaction management, logging, metadata management, and storage [4]. Microsoft Socrates and emerging systems like PolarDB [18] pursue this vision.

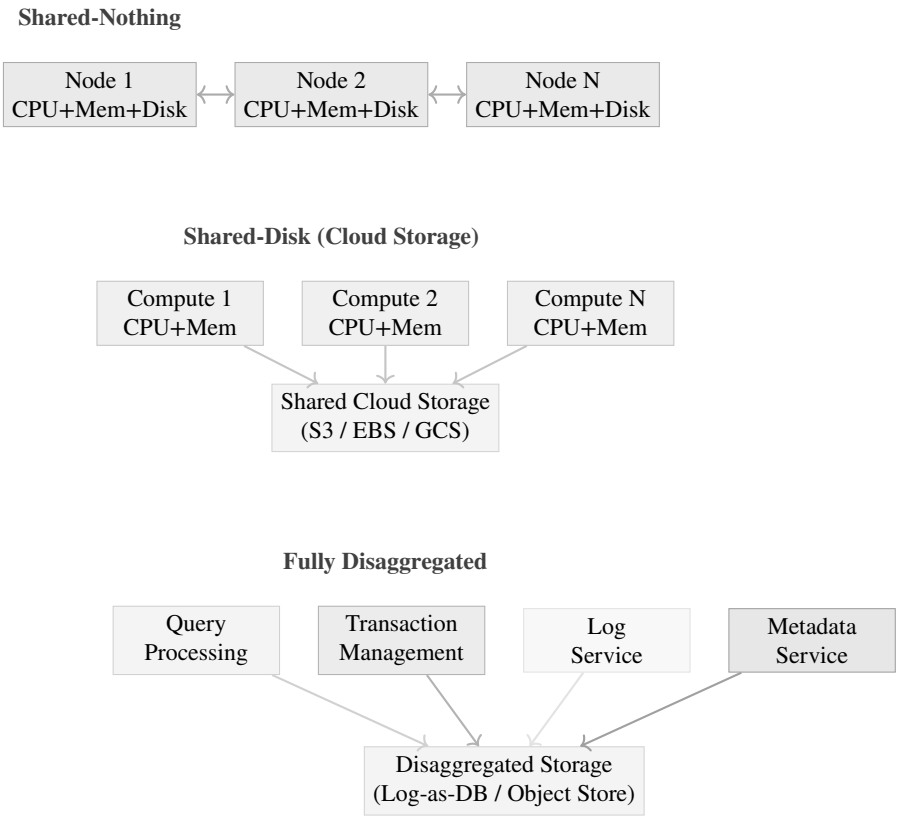


Figure 3: Evolution of cloud database architectures: from shared-nothing clusters to shared-disk (cloud storage) to fully disaggregated microservice architectures.

### 3.4. Data models in the cloud

Cloud platforms support a diverse ecosystem of data models, each optimized for different workload characteristics [38, 39, 40, 41, 42, 43]; early comparisons of the first-generation cloud stores such as SimpleDB and Bigtable already highlighted this model diversity [44]. Table 3 compares the major data models available in cloud database services.

The trend toward *multi-model databases* [45] reflects the desire to avoid managing separate database systems for different data models. Azure Cosmos DB, for instance, supports document, graph, key-value, column-family, and table APIs through a single backend. However, multi-model integration introduces challenges in query optimization, consistency semantics, and performance trade-offs [45].

### 3.5. The data platform continuum: warehouses, lakes, and lakehouses

The cloud data management landscape has converged around three complementary paradigms, illustrated in Figure 4:

**Table 3**
Cloud database data model landscape.

| Data Model | Strengths | Cloud Examples | Typical Use Cases |
|---|---|---|---|
| Relational | ACID, SQL, mature ecosystem | Aurora, Cloud SQL, Azure SQL | OLTP, ERP, financial systems |
| Document | Schema flexibility, nested data | MongoDB Atlas, Cosmos DB, Firestore | Content management, catalogs, user profiles |
| Key-Value | Ultra-low latency, high throughput | DynamoDB, Redis Cloud, Memorystore | Caching, session management, real-time |
| Wide-Column | High write throughput, time-series | Bigtable, Cassandra (Astra), ScyllaDB | IoT, telemetry, event logging |
| Graph | Relationship traversal, pattern matching | Neptune, Neo4j Aura, Cosmos DB (Gremlin) | Social networks, fraud detection, KGs |
| Vector | Similarity search, ANN indexing | Pinecone, Weaviate, Milvus, pgvector | RAG, recommendation, image search |
| Time-Series | Temporal queries, downsampling | TimescaleDB, InfluxDB Cloud, Timestream | Monitoring, financial ticks, sensor data |
| Multi-Model | Multiple models in one engine | Cosmos DB, ArangoDB, SurrealDB | Polyglot persistence, unified access |

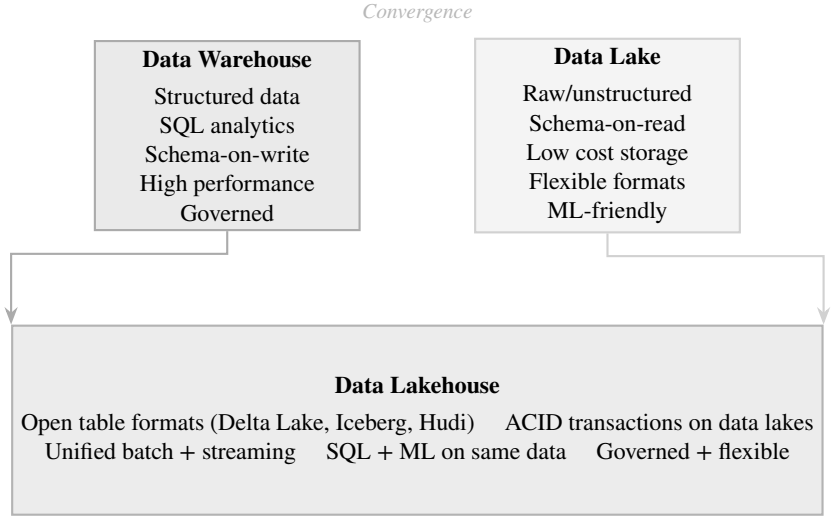


Figure 4: The data platform continuum: data warehouses and data lakes converge into the data lakehouse paradigm.

**Data warehouses** (e.g., Snowflake [3], BigQuery [46], Redshift) provide high-performance SQL analytics over structured, curated data with strong governance and schema enforcement.

**Data lakes** (e.g., Amazon S3 + Athena, Azure Data Lake Storage) offer cost-effective storage for raw, semi-structured, and unstructured data, enabling schema-on-read flexibility for data science and ML workloads.

**Data lakehouses** [16] combine the best of both paradigms by adding ACID transactions, schema enforcement, and data governance to open data lake storage through open table formats such as Delta Lake, Apache Iceberg, and Apache Hudi [10, 47].

**Table 4**
Search terms and digital libraries.

| Library | Search Scope |
|---|---|
| IEEE Xplore | Title, abstract, keywords |
| ACM Digital Library | Title, abstract, full text |
| Springer Link | Title, abstract |
| ScienceDirect (Elsevier) | Title, abstract, keywords |
| Wiley Online Library | Title, abstract |
| PVLDB / VLDB Journal | Title, abstract |
| arXiv | Title, abstract |
| Google Scholar | Title (for supplementary search) |

## 4. Systematic Review Planning and Execution

The systematic literature review (SLR) guidelines established by Kitchenham and Charters [14], which have been widely adopted in software engineering and computer science [48, 49, 50, 51], are followed. This protocol prescribes a three-phase process: (1) *planning*, in which research questions, search strategy, and inclusion/exclusion criteria are defined; (2) *conducting*, in which studies are systematically identified, selected, and their data extracted; and (3) *reporting*, in which the synthesized findings are documented and validated through quality assessment. The selection process is additionally reported using the PRISMA 2020 statement [52], the de facto standard for transparent reporting of systematic reviews.

### 4.1. Search strategy

Eight major digital libraries and indexing services were searched, as detailed in Table 4.

The search string was constructed by combining cloud database terminology with survey-indicating terms:

```
("cloud database" OR "cloud-native database" OR "DBaaS" OR "database-as-a-service" OR
"serverless database" OR "HTAP" OR "NewSQL" OR "data lakehouse" OR "vector database" OR
"AI4DB" OR "learned index" OR "NoSQL" OR "distributed database" OR "multi-model database"
OR "graph database" OR "edge database" OR "fog database" OR "autonomous database")
AND
("survey" OR "review" OR "tutorial" OR "taxonomy" OR "systematic" OR "state-of-the-art" OR
"overview")
```

### 4.2. Study selection

The inclusion and exclusion criteria listed in Table 5 were applied in a two-phase process: (1) title and abstract screening, and (2) full-text assessment.

### 4.3. Quality assessment

Each candidate survey was evaluated against eight quality assessment criteria (QA1–QA8), scored on a 3-point scale (0 = No, 0.5 = Partially, 1 = Yes). Papers scoring below 4 out of 8 were excluded.

### 4.4. Results: study selection and demographics

Figure 5 presents the PRISMA 2020 flow diagram [52] of the selection process. From an initial set of 486 records, 24 surveys were selected for inclusion in the meta-survey after applying all criteria.

**Table 5**
Inclusion and exclusion criteria.

| ID | Criterion |
|---|---|
| *Inclusion Criteria* | |
| IC1 | The paper is a survey, review, tutorial, or systematic mapping study |
| IC2 | The paper focuses primarily on cloud-hosted, cloud-native, or cloud-relevant database technologies |
| IC3 | The paper was published between January 2016 and December 2025 |
| IC4 | The paper is written in English |
| IC5 | The paper is peer-reviewed or published in a recognized venue (including arXiv with significant citations) |
| *Exclusion Criteria* | |
| EC1 | The paper is a primary study (not a survey/review) |
| EC2 | The paper surveys cloud computing in general without database-specific focus |
| EC3 | The paper is a short paper (<6 pages) or workshop position paper |
| EC4 | The paper is a duplicate or substantially overlapping earlier version |
| EC5 | The paper is not accessible in full text |

**Table 6**
Quality assessment checklist for surveyed papers.

| ID | Quality Assessment Question |
|---|---|
| QA1 | Are the survey objectives clearly stated? |
| QA2 | Is the scope and coverage of the survey well-defined? |
| QA3 | Is the search/selection methodology described or implied? |
| QA4 | Does the survey provide a taxonomy or classification framework? |
| QA5 | Are the reviewed primary studies adequately described? |
| QA6 | Does the survey identify open challenges or future directions? |
| QA7 | Is the survey published in a recognized venue (journal, top conference, or high-citation arXiv)? |
| QA8 | Is the survey sufficiently comprehensive (≥30 references)? |

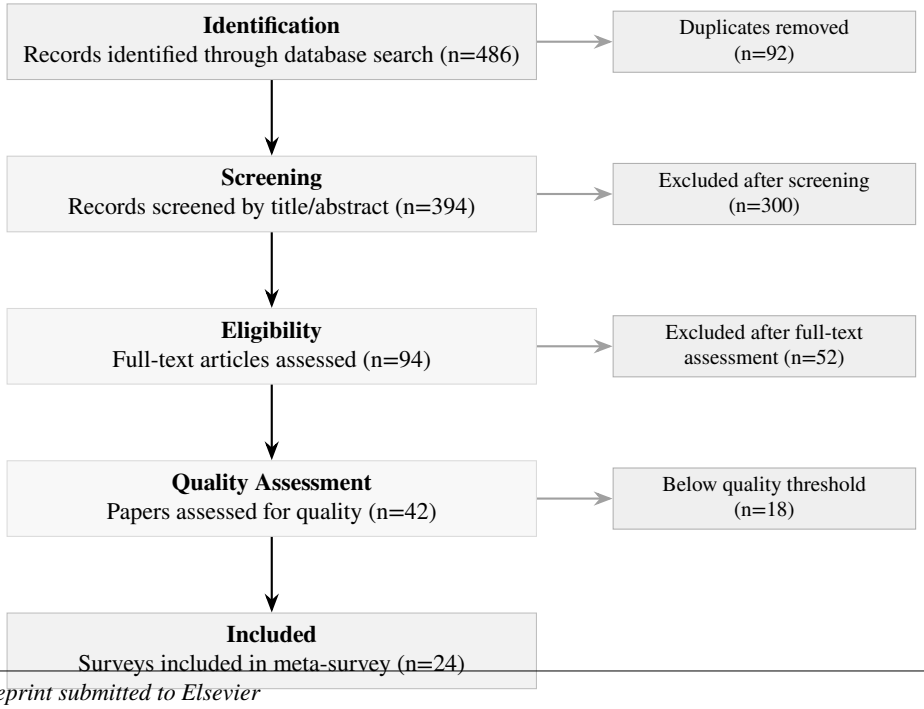


Figure 5: PRISMA 2020 flow diagram of the study selection process [52].

Figure 6 shows the distribution of selected surveys by publication year, revealing an upward trend that intensifies markedly in 2024.

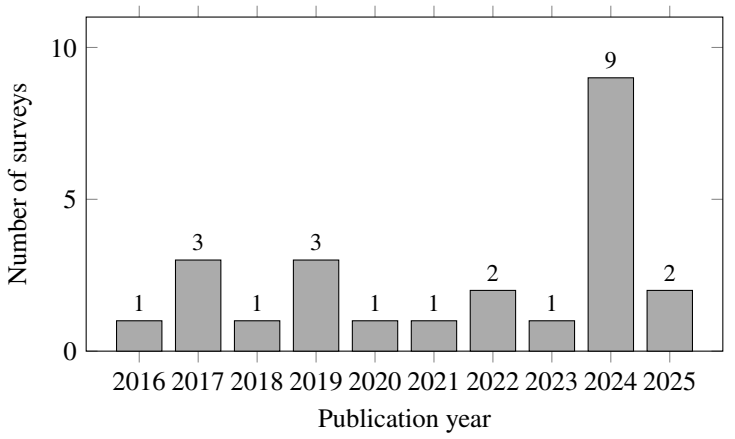


Figure 6: Distribution of the 24 selected surveys by publication year (2016–2025).

Figure 7 shows the distribution by primary topic domain (each survey is assigned a single primary topic, so the counts sum to 24). NoSQL/NewSQL systems are the most frequently surveyed topic, reflecting the maturity and breadth of that body of literature.

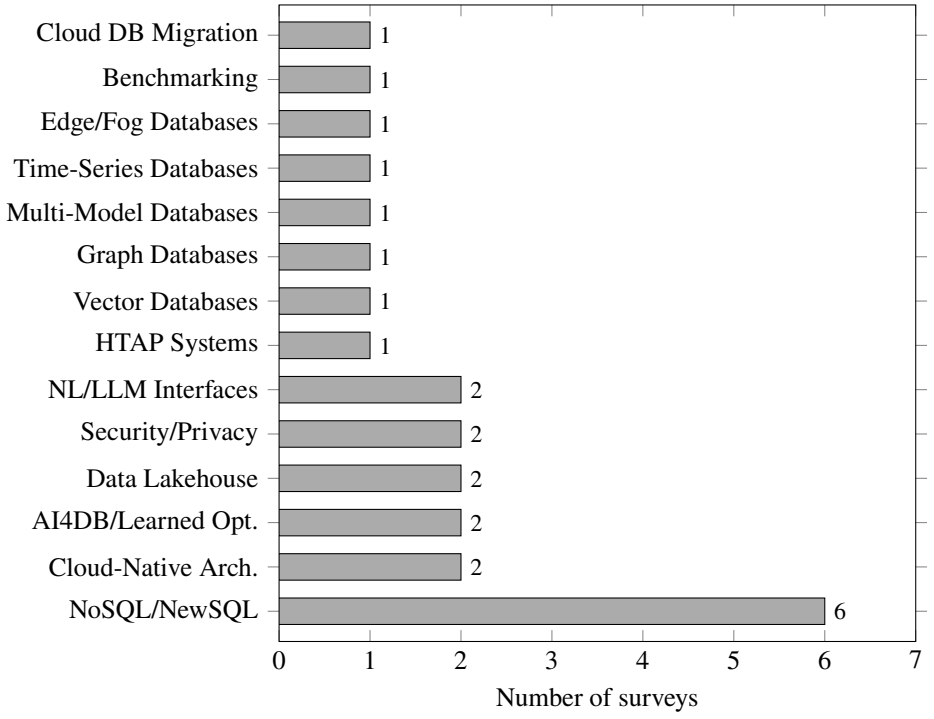


Figure 7: Distribution of the 24 selected surveys by primary topic domain.

Figure 8 shows the distribution by venue type. Journal publications account for the majority, followed by conference proceedings and arXiv preprints.

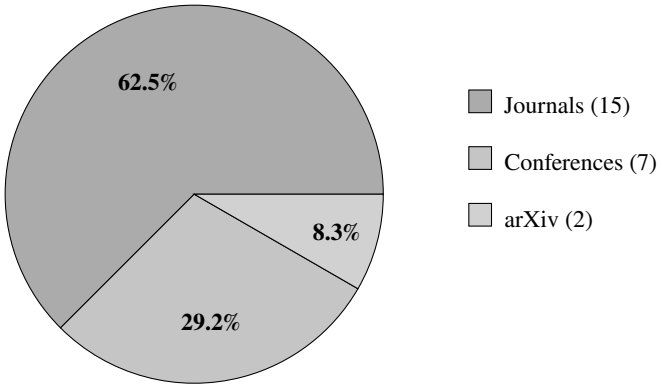


Figure 8: Distribution of the 24 selected surveys by venue type.

### 4.5. Threats to validity

As with any systematic review, this meta-survey is subject to validity threats, which are mitigated as follows.

**Construct validity.** The search string and digital-library selection may not capture every relevant survey, particularly those using non-standard terminology. This threat is mitigated by combining a broad set of cloud database terms with multiple survey-indicating keywords (Section 4.1) and by supplementing database searches with backward and forward snowballing on included papers.

**Internal validity.** Study selection, quality assessment, and data extraction were performed by a single reviewer, introducing a risk of subjective bias and a lack of inter-rater agreement. To reduce this threat, explicit inclusion/exclusion criteria (Table 5) and a documented quality-assessment rubric (Table 6) were applied uniformly, and borderline decisions were re-examined in a second pass. Every included survey was additionally verified against a resolvable DOI or arXiv identifier and a retained full-text copy, so that the catalog contains only confirmable publications. The absence of a second independent reviewer nonetheless remains a limitation.

**External validity.** As a survey of surveys, the findings inherit the scope and currency limitations of the primary surveys; rapidly evolving topics (e.g., LLM–database convergence) may be underrepresented relative to their current activity. The deliberately curated corpus of 24 confirmable surveys favors verifiability over exhaustive coverage, so some relevant but harder-to-verify or paywalled surveys may be absent. The synthesis is therefore intended as a structured map of the field rather than an exhaustive account of every primary system.

**Conclusion validity.** The challenge taxonomy and the consensus/divergence synthesis (Table 8) involve interpretive judgment. These are grounded in explicit per-challenge evidence and cross-referenced to the surveyed domains in Table 7 to keep the reasoning traceable.

## 5. Discussion—Challenges and Research Directions

This section presents the core contribution of this meta-survey: a comprehensive, structured analysis of the challenges and research directions identified across the 24 surveyed papers. Following the dual-theme approach of Saeed and Omlin [51], the discussion is organized into two complementary themes:

1. **Theme 1: Cross-cutting challenges** (Section 5.1)—eighteen recurring challenges that transcend individual system categories.
2. **Theme 2: Lifecycle-phase challenges** (Section 5.2)—challenges organized by the five phases of the cloud database lifecycle.

Figure 9 provides a high-level overview of the dual-theme taxonomy.

### 5.1. Theme 1: Cross-cutting challenges

Eighteen cross-cutting challenges that recur across multiple cloud database survey domains are identified and organized, in Figure 10, into four clusters—*foundational*, *data-architecture*, *operational*, and *emerging* concerns. Each challenge is discussed as a short narrative that frames the problem, summarizes the current state reported across

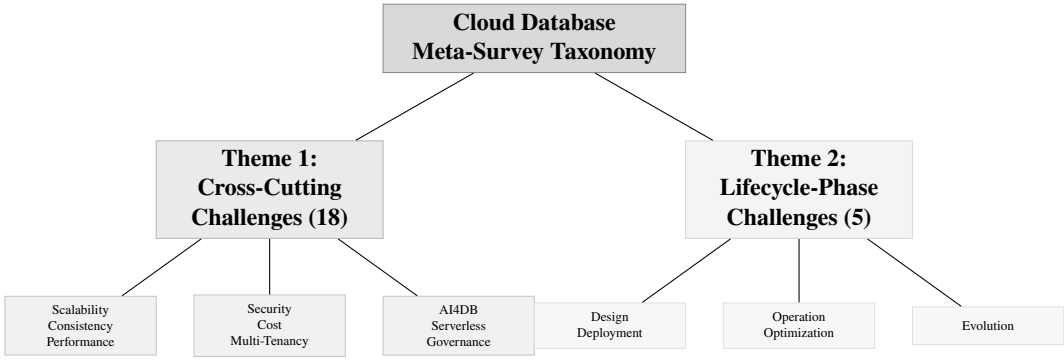


Figure 9: High-level taxonomy of the cloud database meta-survey, organized into two complementary themes.

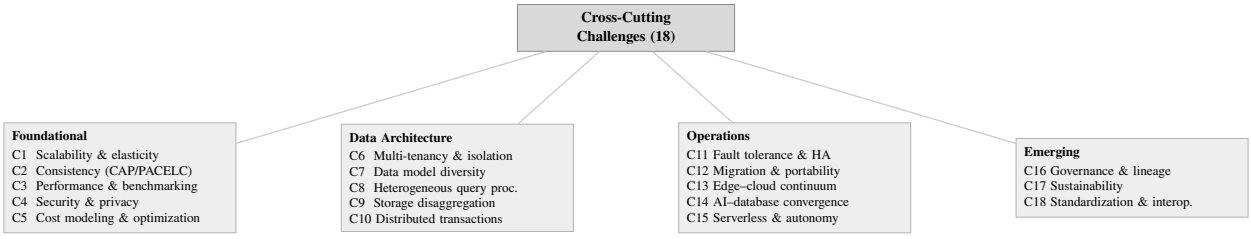


Figure 10: The eighteen cross-cutting challenges, organized into four clusters.

the surveyed papers (including points of consensus and divergence), and closes with a brief statement of the open problems and research directions—highlighting data- and knowledge-engineering (DKE) relevance—that the surveys leave unresolved.

Three consolidated views support the discussion. Table 7 maps each challenge to the survey domains that address it, revealing coverage density and thematic clusters; Table 8 synthesizes the key points of agreement and divergence across surveyed papers; and Table 9 shows how attention to each challenge has evolved over the study window. Detailed discussion follows in Sections 5.1.1–5.1.18.

Table 7: Cross-cutting challenge coverage across consolidated survey domains. Symbols: • = primary focus, ○ = substantive discussion, · = mentioned. The rightmost column (*n*) counts domains addressing each challenge.

| # | Challenge | Cloud-Native | HTAP | NoSQL / NewSQL | AI4DB / Learned | Vector / Graph / Multi-Model | Lakehouse | Time-Series | Edge / Fog | NL / LLM–DB | Security / Privacy | Benchmarking | Migration | *n* |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Foundational* | | | | | | | | | | | | | | |
| C1 | Scalability & elasticity | • | ○ | • | · | | ○ | ○ | ○ | | | ○ | | 8 |
| C2 | Consistency & CAP/PACELC | • | • | • | | · | | | ○ | | | | | 5 |
| C3 | Performance & benchmarking | • | ○ | ○ | • | | ○ | | | | | • | | 6 |
| C4 | Security & privacy | ○ | | · | | | | | ○ | | • | | · | 5 |
| C5 | Cost modeling & optimization | ○ | | | | | ○ | | | | | • | ○ | 4 |
| *Data Architecture* | | | | | | | | | | | | | | |
| C6 | Multi-tenancy & isolation | • | | | | | | | | | ○ | ○ | | 3 |
| C7 | Data model diversity | | · | • | | • | ○ | ○ | | | | | | 5 |
| C8 | Heterogeneous query processing | ○ | • | · | ○ | ○ | • | | | ○ | | | | 7 |
| C9 | Storage disaggregation | • | ○ | | | | • | | | | | | | 3 |
| C10 | Distributed transactions | • | • | • | | · | | | · | | | | | 5 |
| *Operations* | | | | | | | | | | | | | | |
| C11 | Fault tolerance & HA | • | | ○ | | | | | ○ | | | | | 3 |
| C12 | Migration & portability | ○ | | · | | | | | | | | | • | 3 |
| C13 | Edge–cloud continuum | · | | | | | | | • | | · | | | 3 |
| C14 | AI–database convergence | ○ | | | • | · | ○ | | | • | | | | 5 |
| C15 | Serverless & autonomy | • | | | ○ | | | | | | | · | | 3 |
| *Emerging* | | | | | | | | | | | | | | |
| C16 | Governance & lineage | · | | | | | ○ | | | · | • | | · | 5 |
| C17 | Sustainability | · | | | · | | | | | | | ○ | | 3 |

*Continued on next page*

*Table 7 continued*

| # | Challenge | Cloud-Native | HTAP | NoSQL / NewSQL | AI4DB / Learned | Vector / Graph / Multi-Model | Lakehouse | Time-Series | Edge / Fog | NL / LLM–DB | Security / Privacy | Benchmarking | Migration | *n* |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C18 | Standardization & interoperability | ∘ | | ∘ | | • | ∘ | | | · | | | ∘ | 6 |

Table 8: Synthesis of cross-cutting challenges: key similarities (consensus) and differences (divergences) across the 24 surveyed papers.

| # | Challenge | Relevant Surveys | Key Similarities (Consensus) | Key Differences (Divergences) |
|---|---|---|---|---|
| C1 | Scalability & elasticity | S1, S2, S6, S7, S14, S24 | All surveys agree that disaggregated storage enables elastic scaling; horizontal scale-out is preferred over vertical scaling; auto-scaling is a baseline cloud capability. | Cloud-native surveys (S1, S24) focus on component-level elasticity; NoSQL surveys (S6, S7) emphasize partition-based data scaling; HTAP surveys (S2) and edge/fog surveys (S14) stress elasticity under mixed and intermittently connected workloads, respectively. |
| C2 | Consistency & CAP/PACELC | S1, S2, S6, S7, S8, S14 | CAP/PACELC trade-offs are universally acknowledged; all surveys recognize a spectrum from strong to eventual consistency; configurable consistency is a key cloud differentiator. | NoSQL surveys (S6, S7, S8) accept eventual consistency as pragmatic; HTAP surveys (S2) insist on strong consistency for transactional correctness; cloud-native surveys (S1) present tunable consistency as an application-level choice. |
| C3 | Performance & benchmarking | S1, S2, S4, S5, S9, S22 | Cloud-specific factors (network latency, multi-tenancy, cold starts) complicate optimization; existing benchmarks (TPC-C/H, YCSB) are widely considered inadequate for cloud settings. | AI4DB surveys (S4, S5) emphasize learned optimizers and indexes; benchmarking surveys (S9, S22) focus on measurement methodology and fairness; cloud-native surveys (S1) focus on disaggregation overhead. |
| C4 | Security & privacy | S1, S14, S20, S21 | Encryption at rest and in transit is a baseline requirement; the shared responsibility model is universally acknowledged; GDPR compliance is a common concern. | Security surveys (S20, S21) deeply analyze threat models and advanced techniques (homomorphic encryption, confidential computing); cloud-native surveys (S1) treat security as one concern among many; edge/fog surveys (S14) emphasize privacy-preserving aggregation. |
| C5 | Cost modeling & optimization | S1, S15, S22, S23 | Cost is a primary driver of cloud adoption; pricing complexity (10–100× variation) is widely noted; pay-per-use models are a defining benefit. | Benchmarking surveys (S22) focus on total cost of ownership comparison; lakehouse surveys (S15) weigh storage–compute cost trade-offs; migration surveys (S23) highlight hidden migration and lock-in costs; cloud-native surveys (S1) note multi-dimensional pricing complexity. |
| C6 | Multi-tenancy & isolation | S1, S20, S22 | Noisy-neighbor performance interference is universally recognized; the trade-off between resource sharing efficiency and isolation is a fundamental tension. | Cloud-native surveys (S1) focus on architectural isolation levels (shared-nothing to shared-table); security surveys (S20) emphasize tenant data isolation; benchmarking surveys (S22) measure interference under controlled conditions. |

*Continued on next page*

*Table 8 continued*

| # | Challenge | Relevant Surveys | Key Similarities (Consensus) | Key Differences (Divergences) |
|---|---|---|---|---|
| C7 | Data model diversity | S3, S6, S7, S8, S12, S13, S17 | Polyglot persistence is the operational reality; cross-model query support is needed; multi-model databases are gaining traction. | NoSQL surveys (S6, S7, S8) favor model-specific optimization; multi-model surveys (S12) advocate unified backends; specialized surveys (S3, S13, S17) argue that model-native operations (vector, graph, time-series) cannot be efficiently reduced to a single backend. |
| C8 | Heterogeneous query processing | S1, S2, S12, S15, S16, S18 | Federated query processing across engines is increasingly necessary; predicate pushdown is the baseline optimization technique. | HTAP surveys (S2) focus on OLTP–OLAP workload routing; lakehouse surveys (S15, S16) emphasize open-format access layers; multi-model surveys (S12) and natural-language interface surveys (S18) address cross-model and intent-driven query routing. |
| C9 | Storage disaggregation | S1, S2, S15, S16 | Compute-storage separation is the defining cloud-native architectural principle; write amplification reduction is a key benefit; independent layer scaling is a primary motivation. | Cloud-native surveys (S1) focus on log-as-database architectures (Aurora, PolarDB); lakehouse surveys (S15, S16) emphasize open table formats; HTAP surveys (S2) address separate analytical and transactional storage tiers. |
| C10 | Distributed transactions | S1, S2, S6, S7, S8 | 2PC and consensus protocols (Paxos/Raft) are the dominant approaches; the consistency–latency trade-off is universally acknowledged. | Cloud-native surveys (S1) cite clock synchronization innovations (TrueTime, HLC); NoSQL surveys (S6, S7, S8) accept weaker guarantees (CRDTs, eventual consistency); HTAP surveys (S2) focus on OLTP–OLAP transaction isolation. |
| C11 | Fault tolerance & HA | S1, S6, S8, S14 | Multi-AZ replication is the baseline approach; automated failover is expected; RTO/RPO targets drive architectural decisions. | Cloud-native surveys (S1) focus on quorum protocols (e.g., Aurora's 4/6 write quorum); NoSQL surveys (S6, S8) emphasize tunable replication factors; edge/fog surveys (S14) must handle intermittent connectivity. |
| C12 | Migration & portability | S1, S6, S23 | Migration is complex, error-prone, and labor-intensive; schema heterogeneity is the primary technical barrier; vendor lock-in is widely criticized. | Migration surveys (S23) focus on methodology and zero-downtime techniques; cloud-native surveys (S1) treat migration as a cloud adoption path; NoSQL surveys (S6) highlight data model mismatch challenges. |
| C13 | Edge–cloud continuum | S1, S14, S20 | Data placement across edge–fog–cloud tiers requires intelligent policies; intermittent connectivity is the defining constraint. | Edge/fog surveys (S14) focus on resource-constrained processing and synchronization; cloud-native surveys (S1) view edge as a cloud extension; security surveys (S20) emphasize privacy-preserving edge aggregation. |

*Continued on next page*

*Table 8 continued*

| # | Challenge | Relevant Surveys | Key Similarities (Consensus) | Key Differences (Divergences) |
|---|---|---|---|---|
| C14 | AI–database convergence | S1, S4, S5, S18, S19 | Both AI4DB and DB4AI directions are growing rapidly; production deployment of learned components remains limited; training data management is a shared challenge. | AI4DB surveys (S4, S5) focus on individual learned components (indexes, optimizers); natural-language and retrieval-augmented surveys (S18, S19) address LLM-driven querying and retrieval; cloud-native surveys (S1) view AI as a system enhancement. |
| C15 | Serverless & autonomy | S1, S4 | Serverless abstracts infrastructure management; cold-start latency is a known limitation; pay-per-query suits variable workloads. | Cloud-native surveys (S1) treat serverless as one deployment model and catalog autoscaling mechanisms; AI4DB surveys (S4) envision full-stack autonomous databases that self-tune beyond the serverless abstraction. |
| C16 | Governance & lineage | S1, S15, S18, S20, S23 | End-to-end lineage tracking is increasingly important; regulatory compliance (GDPR, CCPA) drives governance adoption; metadata management is fragmented across tools. | Security surveys (S20) focus on access control and audit logging; lakehouse surveys (S15) emphasize unified data cataloging; migration surveys (S23) track lineage across system transitions; cloud-native (S1) and NL-interface (S18) surveys note metadata's role in querying. |
| C17 | Sustainability | S1, S4, S22 | Energy-proportional computing is an aspirational goal; carbon-aware data placement is an emerging research direction. | Cloud-native surveys (S1) note the energy implications of disaggregation; AI4DB surveys (S4) flag the training cost of learned components; benchmarking surveys (S22) propose adding energy and carbon metrics; most other survey domains do not yet address sustainability. |
| C18 | Standardization & interoperability | S3, S6, S7, S12, S13, S18, S23 | Vendor lock-in is widely criticized; SQL remains the primary query lingua franca; open formats (Iceberg, Delta Lake, Hudi) are gaining traction. | Graph and specialized surveys (S3, S13) highlight query-language fragmentation (Cypher, SPARQL, GQL); NoSQL surveys (S6, S7) focus on API heterogeneity; multi-model (S12) and natural-language interface (S18) surveys push unified access, while migration surveys (S23) stress portable formats. |

Table 9: Temporal attention to each cross-cutting challenge: the number of surveyed papers, by publication year, whose primary domain substantively (●/○) addresses the challenge. Cell shading is proportional to the count; attention concentrates in 2024, mirroring the overall publication trend (Figure 6).

| # | Challenge | 2016 | 2017 | 2018 | 2019 | 2020 | 2021 | 2022 | 2023 | 2024 | 2025 | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C1 | Scalability & elasticity | | 2 | 1 | | 1 | | 1 | | 7 | 2 | 14 |
| C2 | Consistency & CAP/PACELC | | 1 | 1 | | 1 | | | | 5 | 2 | 10 |
| C3 | Performance & benchmarking | | 1 | 1 | | 1 | | 2 | 1 | 6 | 2 | 14 |
| C4 | Security & privacy | | 1 | | 1 | | 1 | | | 2 | | 5 |
| C5 | Cost modeling & optimization | 1 | 1 | | | | | 1 | | 3 | | 6 |
| C6 | Multi-tenancy & isolation | | 1 | | 1 | | 1 | | | 2 | | 5 |
| C7 | Data model diversity | | 2 | 1 | 1 | 1 | | 1 | | 4 | 2 | 12 |
| C8 | Heterogeneous query processing | | 2 | | 2 | | | 2 | 1 | 5 | | 12 |
| C9 | Storage disaggregation | | 1 | | | | | 1 | | 3 | | 5 |
| C10 | Distributed transactions | | 1 | 1 | | 1 | | | | 4 | 2 | 9 |
| C11 | Fault tolerance & HA | | 1 | 1 | | 1 | | | | 4 | 2 | 9 |
| C12 | Migration & portability | 1 | 1 | | | | | | | 1 | | 3 |
| C13 | Edge–cloud continuum | | | | | | | | | 1 | | 1 |
| C14 | AI–database convergence | | 1 | | 1 | | | 2 | 1 | 3 | | 8 |
| C15 | Serverless & autonomy | | 1 | | | | | 1 | 1 | 1 | | 4 |
| C16 | Governance & lineage | | | | 1 | | 1 | 1 | | 1 | | 4 |
| C17 | Sustainability | | | | | | | | | 1 | | 1 |
| C18 | Standardization & interop. | 1 | 2 | 1 | 1 | 1 | | 1 | | 5 | 2 | 14 |
| **Total (challenge-mentions)** | | **3** | **19** | **7** | **8** | **7** | **3** | **13** | **4** | **58** | **14** | **136** |

#### *5.1.1. Scalability and elasticity*

Scalability—handling growing workloads by adding resources—and elasticity—provisioning and releasing resources in response to demand—are the foundational promises of the cloud, and every surveyed cloud-native and NoSQL review treats them as first-order concerns [6, 11]; efforts to quantify elasticity empirically date back to the first wave of cloud data stores [53]. In practice, cloud-native engines such as Aurora [2] and Snowflake [3] achieve storage elasticity through compute–storage disaggregation, while compute elasticity is delivered by auto-scaling and replication [54]; serverless offerings push this further by scaling to zero between requests [28]. The surveyed literature nonetheless converges on a shared limitation: today's elasticity is coarse-grained, operating at the level of whole instances rather than individual components, so that *fine-grained* elasticity—independently scaling a single query operator, the transaction manager, or the log service—remains largely unrealized [6].

Several obstacles recur across the reviews. Cold-start latency undermines the responsiveness that makes scale-to-zero attractive [28]; state migration during scale-out and scale-in introduces transient performance degradation; and predictive autoscaling depends on workload-forecasting models that are still maturing [55]. Cross-component elasticity in fully disaggregated architectures is the least understood of all, because the interactions among independently scaled layers are difficult to model.

**Open problems and research directions.** The central open problem is a principled account of *elastic data architectures* that links workload patterns, resource allocation, performance guarantees, and cost. From a data- and knowledge-engineering standpoint, ontological models of cloud-database configurations and their scaling behavior could turn today's heuristic autoscaling into reasoning over an explicit model, enabling predictable, component-level elasticity.

#### *5.1.2. Consistency, availability, and partition tolerance revisited*

The CAP theorem [56] and its PACELC refinement [57] have shaped distributed database design for over a decade, but the surveyed work shows that geo-distribution, multi-region deployment, and the proliferation of configurable consistency levels give these trade-offs new dimensions in the cloud [58]. Cloud databases now expose a spectrum of guarantees: Spanner provides external consistency via TrueTime [37], whereas systems such as Cosmos DB offer several intermediate levels from strong to eventual. Surveys of NoSQL stores [11, 12] and of HTAP systems [7] consistently single out consistency management as a defining difficulty, and they disagree on how much weakening is acceptable—NoSQL reviews accept eventual consistency as pragmatic, while HTAP and NewSQL reviews insist on strong guarantees for transactional correctness.

What the surveys leave open is largely a matter of reasoning and tooling. Developers struggle to predict how a given consistency level affects application correctness [58]; adaptive consistency that adjusts to workload requirements is frequently proposed but rarely deployed; and cross-model consistency—reconciling, say, document and graph views of the same data—is barely studied in multi-model settings [45].

**Open problems and research directions.** Bridging the gap between formal consistency models and everyday development calls for knowledge-based specifications of consistency *contracts*—machine-checkable statements of the guarantees a service offers and an application requires—so that compatibility can be verified rather than assumed, a natural fit for formal-methods and DKE techniques.

#### *5.1.3. Performance optimization and benchmarking*

Performance optimization in the cloud spans query optimization, indexing, caching, resource allocation, and workload management, and benchmarking supplies the empirical ground truth against which systems are compared [59]. The AI4DB and cloud-native reviews document substantial progress on the optimization side—learned query optimization [60], learned indexes [61], and automated knob tuning [62]—yet they emphasize that cloud-specific factors such as network latency, disaggregation overhead, multi-tenant interference, and cold starts confound techniques designed for single-node systems [9, 6]. On the measurement side, the benchmarking reviews are unanimous that classical benchmarks (TPC-C, TPC-H, YCSB) fail to capture cloud realities such as elasticity, pay-per-use pricing, and variable network performance [59, 63, 64, 65].

The unresolved issues follow directly. Cloud-aware benchmarks that incorporate elasticity, multi-tenancy, and pricing are still lacking; fair cross-provider comparison is confounded by differing hardware, pricing, and service-level agreements; and end-to-end performance in disaggregated stacks emerges from complex interactions among independently scaled layers that no current benchmark isolates.

**Open problems and research directions.** A promising direction is a *benchmark knowledge base* that formally records configurations, workload characteristics, hardware, and results in a structured, queryable representation; making benchmark provenance explicit would render cloud-database evaluations reproducible and genuinely comparable—an evaluation problem with a strong knowledge-engineering core.

#### *5.1.4. Security and privacy*

Relative to on-premises systems, cloud databases face an enlarged attack surface arising from shared infrastructure, network-based access, and the need to trust the provider [66, 67, 68]. The security-focused surveys catalog the established defenses—encryption at rest and in transit, access control, audit logging, and vulnerability management [69, 70, 71, 72]—and the more advanced techniques that target computation over protected data, including homomorphic encryption for querying ciphertext [73, 74], architectures giving clients distributed, concurrent access to encrypted cloud databases without a trusted broker [75], private-database models for outsourced data [76], secure multi-party computation, and confidential computing in hardware enclaves. The reviews differ in emphasis: dedicated security surveys analyze threat models in depth, whereas cloud-native reviews treat security as one concern among many and edge-oriented work stresses privacy-preserving aggregation close to the data source.

Across the literature the same hard problems persist: the performance overhead of encryption remains prohibitive for analytics; key management across multi-cloud and hybrid deployments is error-prone; the "honest but curious" provider is difficult to defend against without heavy penalties; and reconciling GDPR's right to erasure with analytic utility is unsolved.

**Open problems and research directions.** *Privacy ontologies* that formally model data sensitivity, access policies, and regulatory obligations would let compliance be checked and enforced automatically across heterogeneous cloud deployments, turning today's manual, document-driven compliance into machine-reasoned policy—squarely a DKE contribution.

#### *5.1.5. Cost modeling and optimization*

Cloud databases bill along many dimensions at once—compute hours, storage, I/O operations, network egress, backups, and premium features such as encryption or multi-region replication—which makes cost both a primary adoption driver and a persistent source of surprise [26]. Empirical studies show that the cost of the same workload can vary by one to two orders of magnitude depending on configuration [26], and although work on serverless analytics [28] and the cloud-native reviews [6] flag cost as critical, they offer little quantitative guidance. The benchmarking and migration reviews add the complementary observations that total-cost-of-ownership comparison and hidden lock-in costs are rarely modeled rigorously [63, 77].

Consequently, pre-execution cost estimation for complex analytical queries remains unreliable; multi-objective optimization over cost, performance, and consistency lacks mature frameworks; the trade-offs among reserved, on-demand, and spot capacity for database workloads are understudied; and vendor lock-in obscures cross-provider comparison.

**Open problems and research directions.** Formal *cost knowledge models*—ontologies that capture provider pricing structures, workload profiles, and optimization strategies—could support automated, explainable cost optimization, and would give the field a shared vocabulary for reasoning about cloud economics rather than per-provider spreadsheets.

#### *5.1.6. Multi-tenancy and resource isolation*

Multi-tenancy lets providers serve many customers from shared infrastructure, improving utilization and lowering cost, at the price of harder performance isolation, data separation, and fair resource allocation [6]. The surveyed systems realize tenancy at several granularities—shared-nothing (an instance per tenant), shared-process (logical isolation within one engine), and shared-table (row-level security over common tables)—and the reviews repeatedly identify "noisy neighbor" interference, in which one tenant's load degrades another's latency, as the dominant operational hazard [78].

**Open problems and research directions.** Achieving strong performance isolation without dedicating hardware remains difficult; workload-aware scheduling that adapts to heterogeneous tenants is immature; and the fundamental tension between sharing efficiency and isolation guarantees still lacks a principled, cost-aware resolution. Quantitative models that predict interference from tenant workload signatures are a promising avenue.

#### *5.1.7. Data model diversity and multi-model integration*

The cloud has multiplied the data models in routine use—relational, document, graph, key-value, wide-column, vector, and time-series—and applications increasingly need to query across several at once [45, 79, 80, 81]. Multi-model engines such as Cosmos DB, ArangoDB, and SurrealDB expose multiple model interfaces over a single backend, and the NoSQL and multi-model surveys report growing adoption alongside a candid account of the limits: cross-model query optimization and unified transaction semantics remain weak [11, 82, 83, 84, 45]. The reviews also diverge on strategy—NoSQL surveys favor model-specific optimization, multi-model surveys advocate unified backends, and specialized reviews argue that vector, graph, and time-series operations cannot be reduced efficiently to a single engine.

**Open problems and research directions.** The persistent gaps are unified query languages spanning relational, graph, document, and vector operations; cross-model optimization of queries that combine, say, graph traversal with relational joins and similarity search; and schema mapping across models. Using ontologies to provide a single conceptual layer over diverse physical models—*knowledge representation for multi-model data*—is a direct DKE opportunity.

#### *5.1.8. Query processing across heterogeneous engines*

Modern cloud data stacks rarely rely on a single engine; OLTP, OLAP, graph, search, and vector systems must be queried in concert, making federated query processing and data virtualization essential [7, 85]. Engines such as Presto/Trino, BigQuery Omni, and AWS Lake Formation already provide federation, and the HTAP reviews analyze how queries should be routed to the appropriate engine [7]; yet they agree that cross-engine optimization is still rudimentary, seldom going beyond predicate pushdown. The unresolved problems are cost-based optimization across engines with incompatible cost models and capabilities, preserving transactional guarantees across engine boundaries, and the semantic understanding of data needed to route queries intelligently rather than syntactically.

**Open problems and research directions.** *Ontology-mediated query answering* [24] offers a principled basis for posing a single query against many heterogeneous engines through a shared conceptual schema, connecting cloud query federation to a long line of DKE research on data integration.

#### *5.1.9. Storage disaggregation and log-as-database*

Separating compute from storage into independently scalable layers is the defining architectural move of the cloud-native era, and the "log-as-database" pattern pioneered by Aurora—treating the redo log itself as the system of record—pushes intelligence into the storage tier [2, 6]. This design reduced write amplification and enabled fast failover, and systems such as PolarDB [18] and Socrates [4] carry disaggregation further by splitting out additional components. The cloud-native surveys report that while disaggregation simplifies operations and scaling, it introduces its own difficulties in remote-I/O latency, cache coherence, and log management [6].

**Open problems and research directions.** Outstanding issues include masking the latency gap between remote storage and local SSDs, coordinating caches across compute and storage layers, and managing the log at scale through compaction, garbage collection, and efficient replay. Cost- and latency-aware models of where computation should run within a disaggregated stack are a promising research direction.

#### *5.1.10. Transaction processing in distributed cloud environments*

Distributed transaction processing must balance ACID guarantees against performance and scalability, and the surveyed systems span the full design space from strongly consistent protocols (two-phase commit, Paxos/Raft) to relaxed models such as CRDTs and eventual consistency [37, 17]. Spanner demonstrated that globally distributed, strongly consistent transactions are feasible using synchronized clocks [37], and CockroachDB and YugabyteDB provide comparable guarantees with hybrid logical clocks [17]; the NewSQL and HTAP reviews nonetheless report that sustaining strong consistency at high throughput for mixed workloads is still hard [7].

**Open problems and research directions.** The recurring open problems are reducing commit latency for geo-distributed transactions without weakening consistency, resolving conflicts efficiently across regions, and isolating transactional and analytical work within a single HTAP engine so that neither degrades the other.

#### *5.1.11. Fault tolerance, recovery, and high availability*

Cloud databases must survive hardware faults, network partitions, availability-zone outages, and even whole-region failures while preserving durability and minimizing downtime [2, 6]. The prevailing approach is multi-availability-zone replication—typically three-way—with automated failover, exemplified by Aurora's six-replica, three-zone write

quorum [2]. The surveys report that recovery-time objectives of seconds and near-zero recovery-point objectives are attainable within a single region, but remain difficult to guarantee across regions.

**Open problems and research directions.** Multi-region disaster recovery with near-zero RPO and RTO is still open, as is consistent recovery in disaggregated architectures whose state is scattered across components; the surveyed work also notes the absence of chaos-engineering and fault-injection frameworks tailored to cloud databases, which would let resilience be tested systematically rather than anecdotally.

#### *5.1.12. Data migration and cross-cloud portability*

Organizations routinely move databases between on-premises and cloud, between providers, and between services, and the migration survey documents how hard it is to preserve consistency, minimize downtime, and transform schemas along the way [77]. Although every major provider offers a migration service, the reviewed evidence is that migrations remain error-prone and labor-intensive, with schema and data-model mismatches the principal technical barrier and vendor lock-in—through proprietary APIs, formats, and features—compounding the difficulty of any subsequent move [86].

**Open problems and research directions.** The field still lacks zero-downtime migration for large, continuously written databases, automated schema mapping between heterogeneous systems, and portability layers that abstract provider-specific features. *Knowledge-based schema mapping*, which represents schema semantics ontologically to automate translation, ties cloud migration directly to DKE's long-standing strengths in schema integration and data exchange.

#### *5.1.13. Edge–cloud data continuum*

The spread of IoT devices, autonomous vehicles, and smart-city infrastructure has created demand for database capability at the network edge operating in concert with the cloud [13], a pressure recognized early in evaluations of cloud databases for IoT data [87]. The edge-and-fog survey maps architectures that distribute storage and query processing across edge devices, fog nodes, and cloud backends, and notes that systems such as Azure SQL Edge, AWS Outposts, and locality-aware features in CockroachDB address only parts of the problem [13].

**Open problems and research directions.** Maintaining consistency across the edge–fog–cloud continuum under intermittent connectivity, deciding intelligently which data to cache, replicate, or move, processing queries under tight resource constraints on edge devices, and aggregating data privately before it reaches the cloud all remain open. Policy models that reason about placement against latency, privacy, and connectivity constraints are a promising direction.

#### *5.1.14. AI–database convergence (AI4DB and DB4AI)*

The convergence of AI and databases runs in two directions—AI4DB, which uses learning to optimize the database, and DB4AI, which uses database technology to serve AI workloads [9]. The AI4DB surveys document concrete progress in learned query optimization [60], learned indexes [61, 88, 89], automated knob and buffer tuning [62, 90], learned partitioning advisors [91], cardinality estimation, and workload forecasting [55, 92], while DB4AI spans in-database machine learning and optimized data serving for training pipelines. The reviews agree that learned components are rarely yet trusted in production, and that managing their training data is a shared, unsolved concern. The persistent obstacles are deploying learned components with safety guarantees so that they degrade gracefully when a model fails, collecting and curating training data from live workloads, transferring learned behavior across instances and hardware, and—perhaps most important for adoption—explaining why a learned optimizer made a given decision.

**Open problems and research directions.** *Knowledge engineering for self-driving databases*, which couples explicit domain knowledge about database internals with learned models, is a promising route to autonomous systems that are both robust and explainable—an agenda where DKE and the database community meet directly.

#### *5.1.15. Serverless and autonomous database operations*

Serverless databases hide infrastructure entirely, provisioning, scaling, patching, and tuning without operator involvement, and autonomous databases go further by using machine learning to self-diagnose and self-heal [36, 28]. Offerings such as Aurora Serverless v2, the Cosmos DB serverless tier, Neon, and PlanetScale realize the serverless interface, while Oracle Autonomous Database is the most ambitious autonomy effort; studies of serverless data systems report that the model suits variable workloads well but struggles with sustained high throughput and cold-start latency [28].

**Open problems and research directions.** Predictable performance under serverless autoscaling, transparent exposure of the cost–performance trade-off, and full-stack autonomy that unifies self-tuning, self-healing, and self-securing remain open. The cloud-native and AI4DB reviews converge on the view that genuine autonomy will require databases to reason about their own behavior, not merely react—again pointing to a knowledge-grounded approach.

#### *5.1.16. Data governance, provenance, and lineage*

As cloud data ecosystems grow, tracking where data came from (provenance), how it was transformed (lineage), and whether its use complies with policy becomes essential for compliance, debugging, and trust [25]. The governance-related surveys identify a common set of needs—metadata management, data cataloging, access control, quality monitoring, and regulatory compliance—and report that cloud-native tools such as AWS Glue Data Catalog, Azure Purview, and Google Dataplex address them only partially, leaving the landscape fragmented and unstandardized. The unresolved problems are end-to-end lineage that spans heterogeneous services and processing engines, automated checking against regulatory requirements that themselves keep changing, and provenance-aware query processing that records how each result was derived.

**Open problems and research directions.** Representing and linking metadata as a *knowledge graph* over cloud data assets, and reasoning over it, is a natural DKE contribution that would unify cataloging, lineage, and compliance under a single queryable model.

#### *5.1.17. Sustainability and energy efficiency*

Data centers consumed an estimated 1–1.5% of global electricity in 2023, with projections of 3–4% by 2030 as AI and cloud workloads grow, and cloud databases—heavy consumers of compute and storage—contribute directly to that footprint [93]. Work on green computing identifies opportunities in energy-efficient query processing, scheduling that follows renewable-energy availability, and carbon-aware data placement; major providers now report carbon metrics, but the surveyed evidence is that database-level energy optimization is still nascent.

**Open problems and research directions.** Energy-proportional databases that consume resources in line with actual load, carbon-aware placement and replication that favor cleaner grids, and benchmarks that report energy and carbon alongside performance are the principal open directions; sustainability is also the least-addressed challenge across the surveyed domains, marking it as an opportunity rather than a settled topic.

#### *5.1.18. Standardization and interoperability*

The proliferation of cloud database systems, query languages, APIs, and formats has produced an interoperability problem: SQL remains the lingua franca for relational data, but graph databases alone span Cypher, SPARQL, Gremlin, and GQL, and NoSQL systems expose largely proprietary APIs [8]. The surveys note genuine progress—GQL's emergence as an ISO standard, the SQL/PGQ extension for property graphs, and the consolidation of open table formats such as Iceberg, Delta Lake, and Hudi—while cautioning that vendors continue to differentiate through proprietary extensions, keeping true interoperability out of reach.

**Open problems and research directions.** Universal query interfaces spanning relational, graph, document, and vector operations, standardized control-plane APIs for provisioning and monitoring, and open formats and protocols that blunt lock-in are the recurring asks. Because they are fundamentally about shared schemas and vocabularies, these are interoperability problems to which knowledge-engineering methods apply directly.

### 5.2. Theme 2: Challenges by cloud database lifecycle phases

Complementing the cross-cutting perspective, challenges are organized by the five phases of the cloud database lifecycle: design, deployment, operation, optimization, and evolution. Figure 11 illustrates this lifecycle with key challenges mapped to each phase, and Table 10 groups the surveyed papers by the phase(s) they address. The distribution is itself revealing: design and optimization are the most heavily surveyed phases, whereas the *evolution* phase—schema change, technology migration, and the adoption of new paradigms such as vector and lakehouse systems—is addressed by the fewest surveys, marking it as the least mature stage of the lifecycle and a clear opportunity for future work.

#### *5.2.1. Design phase*

The design phase encompasses decisions made before a cloud database is deployed, including schema design, data model selection, workload characterization, and compliance architecture.

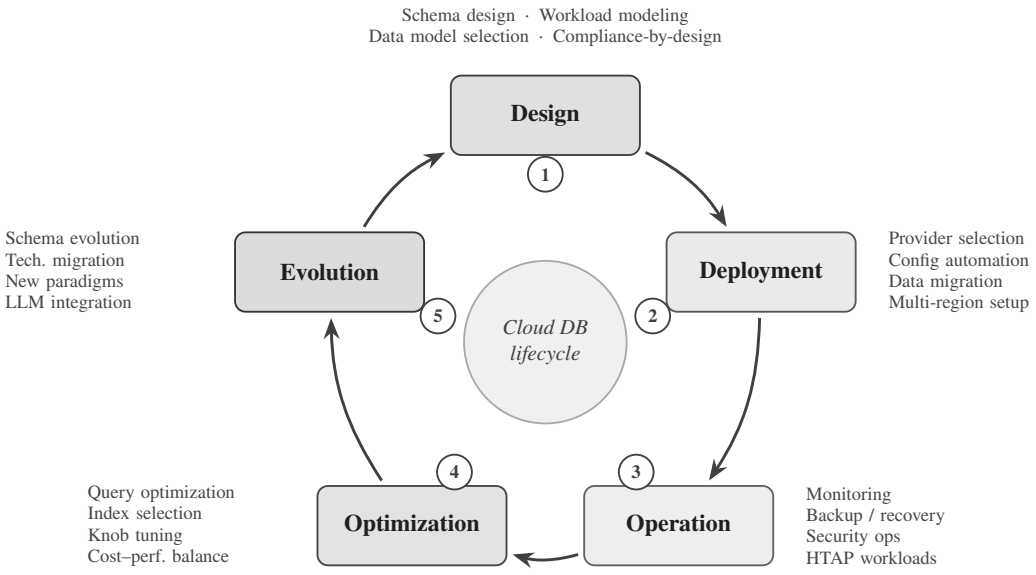


Figure 11: The cloud database lifecycle with key challenges mapped to each phase.

**Table 10**
Surveyed papers grouped by the lifecycle phase(s) they address (derived from the catalog in Table 11; a survey may span several phases, so the counts sum to more than 24).

| Phase | Representative challenges | Surveys | *n* |
|---|---|---|---|
| Design | Schema design, data-model selection, workload characterization, compliance-by-design | S1–S3, S6–S8, S10–S17, S21, S24 | 16 |
| Deployment | Provider selection & lock-in, configuration automation, data ingestion/migration, multi-region setup | S1, S3, S6, S7, S10, S12, S14, S15, S19, S23, S24 | 11 |
| Operation | Monitoring & observability, backup/recovery, security operations, HTAP workload management | S1, S2, S14, S16, S17, S19–S21 | 8 |
| Optimization | Learned query optimization, index/materialized-view selection, knob tuning, cost–performance balance | S1–S5, S8, S9, S11, S13, S18, S22 | 11 |
| Evolution | Schema evolution & online DDL, technology migration, new-paradigm adoption, LLM integration | S4, S23 | 2 |

**Schema design for cloud-native environments.** Traditional schema design principles (normalization, ER modeling) must be adapted for cloud databases where denormalization may improve performance due to the cost of distributed joins [6]. Document and wide-column stores require fundamentally different schema design approaches centered on access patterns rather than entity relationships [11, 94, 95, 96].

**Workload characterization.** Accurate workload modeling is essential for selecting the right cloud database service, configuring resources, and estimating costs. Surveys on AI4DB [9] discuss workload forecasting using ML, but applying these techniques during the design phase—before production data exists—remains challenging.

**Data model selection.** Choosing between relational, document, graph, key-value, wide-column, vector, or multi-model databases requires understanding workload characteristics, query patterns, consistency requirements, and cost constraints. No systematic decision framework exists that accounts for all these dimensions [45].

**Compliance-by-design.** Incorporating regulatory requirements (GDPR, HIPAA, data sovereignty) into the initial database design rather than retrofitting compliance is increasingly recognized as essential [30]. This includes data residency constraints, encryption requirements, and audit trail design.

#### *5.2.2. Deployment phase*

The deployment phase covers provider selection, infrastructure configuration, data ingestion, and multi-region setup.

**Provider selection and vendor lock-in.** Choosing a cloud database provider involves evaluating technical capabilities, pricing, compliance certifications, geographic coverage, and ecosystem integration. Vendor lock-in—through proprietary features, APIs, and data formats—is consistently identified as a top concern across surveys [86, 6].

**Configuration automation.** Infrastructure-as-Code (IaC) tools (Terraform, Pulumi, CloudFormation) enable automated database provisioning, but optimal configuration selection—instance types, storage tiers, replication settings, parameter tuning—often requires deep expertise that is difficult to automate [62].

**Data ingestion and migration.** Initial data loading and ongoing ingestion from diverse sources (operational databases, streaming platforms, file systems) into cloud databases involves ETL/ELT pipeline design, schema mapping, and data quality assurance [77].

**Multi-region deployment.** Deploying databases across multiple geographic regions for latency, availability, and compliance reasons introduces challenges in data synchronization, conflict resolution, and region-aware routing [37].

#### *5.2.3. Operation phase*

The operation phase covers the day-to-day management of running cloud database systems.

**Monitoring and observability.** Cloud databases generate vast amounts of telemetry data (metrics, logs, traces). Effective monitoring requires understanding normal behavior patterns, detecting anomalies, and correlating events across distributed components [55]. Work on self-driving databases [36] emphasizes the need for ML-driven anomaly detection and root cause analysis, and production experience with diagnosing intermittent slow queries at cloud scale shows how difficult root-cause attribution remains [97].

**Backup, recovery, and disaster recovery.** While cloud providers automate basic backups (point-in-time recovery, snapshots), disaster recovery across regions, testing recovery procedures, and meeting stringent RPO/RTO targets remain operational challenges [2].

**Security operations.** Ongoing security management includes access control auditing, encryption key rotation, vulnerability patching, network security configuration, and compliance monitoring. The dynamic nature of cloud environments (auto-scaling, serverless) complicates security posture management [69].

**Real-time workload management (HTAP).** For HTAP systems [7], operational challenges include managing resource contention between transactional and analytical workloads, maintaining freshness guarantees for analytical queries, and preventing analytical queries from degrading OLTP performance.

#### *5.2.4. Optimization phase*

The optimization phase encompasses techniques for improving performance and efficiency of running cloud databases.

**Learned query optimization.** ML-based query optimizers [60, 9] that learn from workload history to generate better query plans represent a paradigm shift from traditional cost-based optimization. However, deploying learned optimizers in production requires addressing safety (avoiding catastrophic regressions), training efficiency, and adaptability to workload changes.

**Automated index and materialized view selection.** Cloud databases increasingly offer automated index recommendation (e.g., Azure SQL automatic tuning, Amazon Redshift automatic materialized views), but surveys [9] report that these systems are often conservative, covering only a fraction of possible optimizations.

**Knob tuning.** Database configuration parameters (buffer pool size, parallelism degree, checkpoint intervals, etc.) significantly impact performance. ML-based tuning systems [62, 90] can explore the high-dimensional configuration space more effectively than manual tuning, but transferring tuning knowledge across different workloads and hardware remains an open problem.

**Cost–performance trade-off optimization.** In cloud environments, performance optimization cannot be divorced from cost. The optimal configuration minimizes cost subject to performance constraints (or maximizes performance subject to budget constraints), requiring multi-objective optimization [26].

#### *5.2.5. Evolution phase*

The evolution phase addresses long-term changes in database systems, technologies, and requirements.

**Schema evolution and online DDL.** Changing database schemas in production without downtime (online DDL) is a critical capability for agile development. Cloud-native databases have improved online DDL support, but complex migrations (column type changes, table splits, data backfills) remain challenging [6].

**Technology migration.** Migrating from one database technology to another (e.g., from a relational database to a document store, or from a legacy system to a cloud-native database) is a major undertaking that surveys identify as poorly supported by existing tools [77].

**Adapting to new paradigms.** The emergence of vector databases for AI workloads [5], the lakehouse paradigm [16], and serverless computing [28] requires existing systems to evolve or be replaced. Understanding when and how to adopt new paradigms is a strategic challenge.

**LLM integration.** The rapid rise of large language models (LLMs) is creating new requirements for cloud databases, from vector storage and retrieval for RAG pipelines to natural language interfaces for database querying and administration (discussed in detail in Section 6).

## 6. Cloud Databases and Large Language Models—A Convergence Case Study

The convergence of cloud databases and large language models (LLMs) represents one of the most dynamic and rapidly evolving areas in modern data management. This section examines this convergence as an illustrative case study, paralleling Saeed and Omlin's [51] treatment of XAI in medicine as a domain-specific application.

### 6.1. LLM-augmented database interfaces

Natural language interfaces to databases (NLIDB) have been a long-standing research goal [98]. The advent of LLMs has dramatically improved text-to-SQL accuracy, with systems like DIN-SQL [99], DAIL-SQL [100], and MAC-SQL [101] achieving over 85% execution accuracy on the Spider benchmark [102].

**Current capabilities.** LLM-based text-to-SQL systems leverage schema-aware prompting, few-shot in-context learning, and chain-of-thought reasoning to translate natural language queries into SQL. Cloud vendors are integrating these capabilities: Amazon Q for databases, Azure AI integration with SQL, and Google Duet AI for BigQuery.

**Open challenges.**

- Complex queries involving multiple joins, subqueries, and window functions remain error-prone.
- Schema understanding for large databases with hundreds of tables and complex relationships.
- Multi-turn conversational interfaces that maintain context across query refinements.
- Handling ambiguity, domain-specific terminology, and user intent clarification.
- Security risks from SQL injection through natural language prompts.

### 6.2. Vector databases as LLM infrastructure

Vector databases have emerged as critical infrastructure for LLM applications, particularly retrieval-augmented generation (RAG) pipelines [5].

**Architecture and workloads.** RAG pipelines embed documents into high-dimensional vectors, store them in vector databases, and retrieve relevant context for LLM prompts based on semantic similarity. This creates unique workload patterns: bulk embedding ingestion, high-throughput approximate nearest neighbor (ANN) search, and hybrid queries combining vector similarity with metadata filtering.

**System landscape.** The vector database market has rapidly diversified, including purpose-built systems (Pinecone, Weaviate, Milvus, Qdrant, Chroma), vector extensions to existing databases (pgvector for PostgreSQL, Elasticsearch vector search), and cloud-native offerings (Amazon OpenSearch vector engine, Azure AI Search, Google Vertex AI Vector Search) [5].

**Open challenges.**

- Scalability and freshness—keeping vector indexes up-to-date as source documents change.
- Multi-modal vector search combining text, image, and structured metadata.

- Quality metrics for retrieval that correlate with downstream LLM task performance.
- Cost optimization for storing and querying billions of high-dimensional vectors.
- Integration with existing cloud database ecosystems (transactions, access control, governance).

### 6.3. LLMs for database administration

LLMs are increasingly applied to automate database administration tasks, including performance diagnosis, query optimization, and anomaly resolution.

**Systems and approaches.** D-Bot [103] uses LLMs for database diagnosis, performing root cause analysis of performance anomalies by querying system views and analyzing metrics. DB-GPT [104] provides an LLM-powered framework for database interaction, supporting text-to-SQL, data analysis, and administration tasks. GPTuner [105] leverages LLMs for database knob tuning, combining GPT-4 knowledge with Bayesian optimization.

**Open challenges.**

- Reliability—LLMs can hallucinate incorrect diagnostic conclusions or generate harmful administration commands.
- Safety guardrails—preventing LLM-driven administration from executing destructive operations.
- Integrating LLM reasoning with structured database knowledge (system catalogs, performance models).
- Evaluation frameworks for LLM-based DBA tools that measure real-world effectiveness.

### 6.4. Challenges and open problems at the convergence

The LLM–database convergence creates several overarching challenges:

1. **Infrastructure co-design.** Jointly optimizing cloud database infrastructure for both traditional workloads and LLM-related workloads (embedding generation, vector search, context retrieval) requires new architectural thinking.
2. **Data governance for AI.** Ensuring that LLM applications respect data access policies, privacy regulations, and audit requirements when retrieving context from cloud databases.
3. **Evaluation and benchmarking.** Standardized benchmarks for vector database performance, text-to-SQL accuracy on real-world schemas, and LLM-based DBA effectiveness are needed.
4. **Cost management.** LLM API calls combined with cloud database queries create complex cost structures that are difficult to predict and optimize.

### 6.5. What do we think? Open questions for cloud databases and LLMs

Looking across the challenges catalogued in Section 5, the cloud-database–LLM convergence raises a set of questions that the surveyed literature has only begun to address. They are offered not as a checklist but as prompts for future work in this domain—and, by analogy, for other data-intensive domains:

- How should a cloud database co-schedule traditional query workloads and LLM-driven workloads (embedding generation, vector search, context retrieval) on shared, elastic infrastructure without one starving the other?
- When an LLM retrieves context from governed cloud data, how can access policies, privacy regulations, and audit obligations be enforced *through* the retrieval path rather than around it?
- What would a faithful benchmark for this convergence measure—vector-search quality, text-to-SQL accuracy on real enterprise schemas, or the end-to-end utility of an LLM-assisted DBA—and who maintains its ground truth?
- Can the reliability of LLM-based administration be bounded well enough to let it act on production systems, and what guardrails make graceful degradation the default when the model is wrong?

- How should the combined cost of LLM inference and cloud-database execution be modeled and optimized as a single objective, given that today they are billed and tuned independently?
- What role can knowledge graphs and ontologies play in grounding LLM reasoning in the actual schema, constraints, and semantics of a cloud database—turning a plausible answer into a correct one?

These questions show how the cross-cutting and lifecycle challenges of Section 5 resurface, sharpened, when cloud databases meet LLMs; the same analysis could be re-instantiated for other domains such as scientific data management or real-time analytics.

## 7. Conclusions

This paper has presented the first systematic meta-survey of cloud database research, synthesizing findings from 24 content-verified survey papers published between 2016 and 2025. Applying the Kitchenham–Charters protocol with PRISMA 2020 reporting, the scattered landscape of cloud database challenges has been consolidated into a structured, dual-theme taxonomy.

For the first theme, eighteen cross-cutting challenges are identified and synthesized: (1) scalability and elasticity; (2) consistency and the CAP/PACELC trade-offs; (3) performance optimization and benchmarking; (4) security and privacy; (5) cost modeling and optimization; (6) multi-tenancy and resource isolation; (7) data-model diversity and multi-model integration; (8) heterogeneous query processing; (9) storage disaggregation and the log-as-database pattern; (10) distributed transaction processing; (11) fault tolerance, recovery, and high availability; (12) data migration and cross-cloud portability; (13) the edge–cloud data continuum; (14) AI–database convergence; (15) serverless and autonomous operation; (16) data governance, provenance, and lineage; (17) sustainability and energy efficiency; and (18) standardization and interoperability. Across these, *scalability*, *consistency*, *security*, *cost*, and *AI–database convergence* recur in the majority of the surveyed domains (Table 9), indicating where progress would benefit the field most broadly.

For the second theme, the challenges are mapped to the five phases of the cloud database lifecycle—design, deployment, operation, optimization, and evolution. Design and optimization are the most thoroughly surveyed phases, whereas evolution (schema change, technology migration, and new-paradigm adoption) is the least mature and most under-served (Table 10), making it a particularly promising target for future work.

Two observations cut across both themes. The convergence of cloud databases with large language models is moving rapidly from research to practice through vector databases, text-to-SQL, and LLM-based administration, yet reliability, governance, and evaluation remain unresolved (Section 6). And from a knowledge-engineering standpoint, cloud databases offer rich openings for DKE techniques—knowledge graphs for metadata and lineage, ontology-mediated integration and query answering, and formal models for consistency, cost, and schema evolution.

Accordingly, this meta-survey is intended to serve three purposes: (1) as a comprehensive reference for researchers entering the cloud database space, providing a structured map of the territory and its open problems; (2) as a strategic guide for practitioners navigating technology selection, deployment, and evolution; and (3) as a bridge between the cloud database and the data-and-knowledge-engineering communities.

**Limitations.** Three limitations qualify these conclusions. First, the corpus is deliberately restricted to surveys that could be content-verified against a resolvable DOI or arXiv identifier, trading breadth for confirmability, so some relevant but harder-to-verify or paywalled surveys may be absent. Second, study selection, quality assessment, and data extraction were performed by a single reviewer (Section 4.5). Third, as a survey of surveys, the findings inherit the currency limitations of the primary surveys, so the fastest-moving topics—most notably LLM–database convergence—may be under-represented relative to their present activity.

### Future work

Based on the analysis of 24 surveys and the challenges discussed in Sections 5 and 6, several priority research directions emerge:

- **Foundational systems.** Cloud-native benchmarks that capture elasticity, multi-tenancy, and pricing; the transition of learned optimizers, learned indexes, and ML-based tuning from prototypes to production-ready, safely-degrading components; and a maturing vector-database ecosystem with standardized APIs and retrieval-quality metrics.

- **Architecture and query processing.** Cross-cloud middleware and portable open formats that reduce vendor lock-in; and federated query processing with consistent transactions across heterogeneous engines and the edge–fog–cloud continuum.
- **Intelligence and autonomy.** Autonomous databases whose learned decisions are explainable and grounded in database knowledge; and intent-driven natural-language interfaces that let users express goals directly while preserving correctness and access control.
- **Governance and sustainability.** Privacy-preserving analytics and end-to-end lineage that scale to regulatory needs; and energy- and carbon-aware database operation that converges the relational, graph, document, vector, and streaming models toward unified, sustainable platforms.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

This study analyzes previously published survey papers, all of which are cited in the manuscript. The systematic review protocol, search strings, screening logs, quality-assessment scores, and complete data-extraction forms are provided as supplementary material to support reproducibility.

## A. Complete List of Surveyed Papers with Classification

Table 11 presents the complete catalog of the 24 surveyed papers, classified by primary topic, year, venue, quality-assessment score, and lifecycle phases covered.

Table 11: Master catalog of surveyed papers (S1–S24). QA = quality-assessment score (out of 8; inclusion threshold ≥4).

| ID | Title (Abbreviated) | Year | Primary Topic | Venue | QA | Lifecycle Phases |
|---|---|---|---|---|---|---|
| S1 | Cloud-Native Databases: A Survey [6] | 2024 | Cloud-Native Arch. | IEEE TKDE | 8 | Design, Deploy, Operate, Optimize |
| S2 | HTAP Databases: A Survey [7] | 2024 | HTAP Systems | arXiv | 7.5 | Design, Operate, Optimize |
| S3 | Survey of Vector Database Management Systems [5] | 2024 | Vector Databases | VLDB Journal | 7.5 | Design, Deploy, Optimize |
| S4 | Database Meets AI: A Survey [9] | 2022 | AI4DB | IEEE TKDE | 8 | Optimize, Evolve |
| S5 | Learned Index: A Comprehensive Experimental Evaluation [89] | 2023 | AI4DB/Learned Idx. | PVLDB | 7 | Optimize |
| S6 | A Survey on NoSQL Stores [11] | 2018 | NoSQL/NewSQL | ACM Comp. Surv. | 8 | Design, Deploy |
| S7 | SQL, NewSQL, and NoSQL Databases: A Comparative Survey [39] | 2020 | NoSQL/NewSQL | IEEE ICICS | 6 | Design, Deploy |
| S8 | Cassandra vs. MongoDB: A Systematic Review [82] | 2024 | NoSQL/NewSQL | IEEE BDAI | 6 | Design, Optimize |
| S9 | SQL and NoSQL in Big Data: An SLR [64] | 2024 | NoSQL/NewSQL | IEEE ICIMCIS | 5.5 | Optimize |
| S10 | NoSQL Databases: A Survey [12] | 2025 | NoSQL/NewSQL | ARIMA | 6 | Design, Deploy |
| S11 | NoSQL Data Warehouse Optimizing Models [94] | 2025 | NoSQL/NewSQL | Big Data Research | 6 | Design, Optimize |
| S12 | Multi-Model Databases: A New Journey [45] | 2019 | Multi-Model DBs | ACM Comp. Surv. | 8 | Design, Deploy |

*Continued on next page*

| ID | Title (Abbreviated) | Year | Primary Topic | Venue | QA | Lifecycle Phases |
|---|---|---|---|---|---|---|
| S13 | Foundations of Query Languages for Graph DBs [8] | 2017 | Graph Databases | ACM Comp. Surv. | 8 | Design, Optimize |
| S14 | Databases in Edge and Fog Environments: A Survey [13] | 2024 | Edge/Fog DBs | ACM Comp. Surv. | 7.5 | Design, Deploy, Operate |
| S15 | From Data Warehouse to Lakehouse: A Comparative Review [10] | 2022 | Data Lakehouse | IEEE BigData | 6.5 | Design, Deploy |
| S16 | Data Lakehouse for Time Series Data: An SLR [47] | 2024 | Data Lakehouse | IEEE BigData | 6.5 | Design, Operate |
| S17 | Open Source Time Series Databases: A Survey [42] | 2017 | Time-Series DBs | BTW | 6 | Design, Operate |
| S18 | Recent NL Interfaces for Databases: A Comparative Survey [98] | 2019 | NL/LLM-Database | VLDB Journal | 7.5 | Optimize |
| S19 | Retrieval-Augmented Generation for LLMs: A Survey [106] | 2024 | NL/LLM-Database | arXiv | 7 | Deploy, Operate |
| S20 | Cloud Computing Security: Threats & Mitigation—An SLR [69] | 2021 | Security/Privacy | IEEE Access | 7 | Operate |
| S21 | Architecture, Security & Performance of DBaaS [72] | 2019 | Security/Privacy | Trans. Emerg. Telecom. Tech. | 6.5 | Design, Operate |
| S22 | DBMS Performance Comparisons: An SLR [63] | 2024 | Benchmarking | J. Syst. Softw. | 7.5 | Optimize |
| S23 | Cloud Migration Process: A Survey [77] | 2016 | Cloud DB Migration | J. Syst. Softw. | 7.5 | Deploy, Evolve |
| S24 | Databases in Cloud Computing: A Literature Review [107] | 2017 | Cloud-Native Arch. | IJITCS | 5.5 | Design, Deploy |

# References


[1] Gartner, Inc., Magic quadrant for cloud database management systems, Gartner Research, accessed: 2025-01-15 (2024).

[2] A. Verbitski, A. Gupta, D. Saha, M. Brahmadesam, K. Gupta, R. Mittal, S. Krishnamurthy, S. Maurice, T. Kharatishvili, X. Bao, Amazon Aurora: Design considerations for high throughput cloud-native relational databases, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2017, pp. 1041–1052. doi:10.1145/3035918.3056101.

[3] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang, A. W. Lee, A. Motivala, A. Q. Munir, S. Pelley, P. Povinec, G. Rahn, S. Triantafyllis, P. Unterbrunner, The Snowflake elastic data warehouse, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2016, pp. 215–226. doi:10.1145/2882903.2903741.

[4] P. Antonopoulos, A. Budovski, C. Diaconu, A. H. Saenz, J. Hu, H. Kodavalla, D. Kossmann, S. Lingam, U. F. Minhas, N. Prakash, V. Purber, H. Qu, C. S. Ravella, K. Reisteter, S. Shrotri, D. Tang, V. Wakade, Socrates: The new SQL server in the cloud, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2019, pp. 1743–1756. doi:10.1145/3299869.3314047.

[5] J. J. Pan, J. Wang, G. Li, Survey of vector database management systems, The VLDB Journal 33 (5) (2024) 1371–1400. doi:10.1007/s00778-024-00864-x.

[6] H. Dong, C. Zhang, G. Li, H. Zhang, Cloud-native databases: A survey, IEEE Transactions on Knowledge and Data Engineering 36 (12) (2024) 7772–7791. doi:10.1109/TKDE.2024.3397508.

[7] C. Zhang, G. Li, J. Zhang, X. Zhang, J. Feng, HTAP databases: A survey, arXiv preprint arXiv:2404.15670 (2024). arXiv:2404.15670.

[8] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. Reutter, D. Vrgoc, Foundations of modern query languages for graph databases, ACM Computing Surveys 50 (5) (2017) 1–40. doi:10.1145/3104031.

[9] X. Zhou, C. Chai, G. Li, J. Sun, Database meets artificial intelligence: A survey, IEEE Transactions on Knowledge and Data Engineering 34 (3) (2022) 1096–1116. doi:10.1109/TKDE.2020.2994641.

[10] A. A. Harby, F. Zulkernine, From data warehouse to lakehouse: A comparative review, in: 2022 IEEE International Conference on Big Data (Big Data), IEEE, 2022, pp. 389–395. doi:10.1109/BigData55660.2022.10020719.

[11] A. Davoudian, L. Chen, M. Liu, A survey on NoSQL stores, ACM Computing Surveys 51 (2) (2018) 1–43. doi:10.1145/3158661.

[12] D. Kpekpassi, D. Faye, NoSQL databases: A survey, Revue Africaine de Recherche en Informatique et Mathématiques Appliquées (ARIMA) (2025). doi:10.46298/arima.13970.

[13] L. M. M. Ferreira, F. Coelho, J. Pereira, Databases in edge and fog environments: A survey, ACM Computing Surveys 56 (11) (2024) 1–40. doi:10.1145/3666001.

[14] B. Kitchenham, S. Charters, Guidelines for performing systematic literature reviews in software engineering, Technical Report EBSE-2007-01, Keele University and Durham University (2007).

[15] Amazon Web Services, Amazon Aurora documentation, https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/, accessed: 2025-01-15 (2023).

[16] M. Armbrust, A. Ghodsi, R. Xin, M. Zaharia, Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics, in: Proceedings of CIDR, 2021.

[17] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Biber, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, P. Mattis, CockroachDB: The resilient geo-distributed SQL database, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2020, pp. 1493–1509. doi:10.1145/3318464.3386134.

[18] W. Cao, Y. Zhang, X. Yang, F. Li, S. Wang, Q. Hu, X. Cheng, Z. Chen, Z. Liu, J. Fang, B. Wang, Y. Wang, H. Sun, Z. Yang, Z. Cheng, S. Chen, J. Wu, W. Hu, J. Zhao, Y. Gao, S. Cai, Y. Zhang, J. Tong, PolarDB serverless: A cloud native database for disaggregated data centers, Proceedings of the ACM SIGMOD International Conference on Management of Data (2021) 2477–2489doi:10.1145/3448016.3457560.

[19] D. Huang, Q. Liu, Q. Cui, Z. Fang, X. Ma, F. Xu, L. Shen, L. Tang, Y. Zhou, M. Huang, W. Wei, C. Liu, J. Zhang, J. Li, X. Wu, L. Song, R. Sun, S. Yu, L. Zhao, N. Cameron, L. Pei, X. Tang, TiDB: A Raft-based HTAP database, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 3072–3084. doi:10.14778/3415478.3415535.

[20] Z. Yang, C. Yang, F. Han, M. Zhuang, B. Yang, Z. Lin, H. Xu, G. Li, L. Ran, S. Wang, Y. Li, R. Pan, L. Guo, OceanBase: A 707 million tpmC distributed relational database system, in: Proceedings of the VLDB Endowment, Vol. 15, 2022, pp. 3385–3397. doi:10.14778/3554821.3554830.

[21] F. Färber, S. K. Cha, J. Primsch, C. Bornhövd, S. Siber, W. Lehner, SAP HANA database: Data management for modern business applications, in: SIGMOD Record, Vol. 40, 2012, pp. 45–51. doi:10.1145/2094114.2094126.

[22] M. Elbattah, M. Roushdy, M. Aref, A.-B. M. Salem, Large-scale ontology storage and query using graph database-oriented approach: The case of Freebase, in: 2015 IEEE Seventh International Conference on Intelligent Computing and Information Systems (ICICIS), IEEE, 2015.

[23] M. E. Coimbra, L. Svitakova, D. Vrgoč, A. P. Francisco, L. Veiga, Survey: Graph databases, arXiv preprint arXiv:2505.24758 (2025). arXiv:2505.24758.

[24] G. Xiao, D. Calvanese, R. Kontchakov, D. Lembo, A. Poggi, R. Rosati, M. Zakharyaschev, Ontology-based data access: A survey, Proceedings of the International Joint Conference on Artificial Intelligence (2018) 5511–5519.

[25] A. Halevy, F. Korn, N. F. Noy, C. Olston, N. Polyzotis, S. Roy, S. E. Whang, Goods: Organizing Google's datasets, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2016, pp. 795–806. doi:10.1145/2882903.2903730.

[26] V. Leis, M. Kuschewski, Towards cost-optimal query processing in the cloud, in: Proceedings of the VLDB Endowment, Vol. 14, 2021, pp. 1606–1612. doi:10.14778/3461535.3461549.

[27] V. Mateljan, D. Čišić, D. Ogrizović, Cloud database-as-a-service (DaaS) – ROI, in: Proceedings of the 33rd International Convention MIPRO, IEEE, 2010.

[28] I. Müller, R. Marroquín, G. Alonso, Lambada: Interactive data analytics on cold data using serverless cloud infrastructure, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2020, pp. 115–130. doi:10.1145/3318464.3389758.

[29] European Parliament and Council of the European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation), Official Journal of the European Union (2016).

[30] E. Politou, E. Alepis, C. Patsakis, Forgetting personal data and revoking consent under the GDPR: Challenges and proposed solutions, Journal of Cybersecurity 4 (1) (2018) tyy001. doi:10.1093/cybsec/tyy001.

[31] C. Curino, E. P. Jones, R. A. Popa, N. Malviya, E. Wu, S. Madden, H. Balakrishnan, N. Zeldovich, Relational cloud: A database-as-a-service for the cloud, in: Proceedings of CIDR, 2011.

[32] W. Lehner, K.-U. Sattler, Database as a service (DBaaS), in: 2010 IEEE 26th International Conference on Data Engineering (ICDE), IEEE, 2010, pp. 1216–1217.

[33] W. A. Shehri, Cloud database database as a service, International Journal of Database Management Systems (IJDMS) 5 (2) (2013) 1–12. doi:10.5121/ijdms.2013.5201.

[34] K. Grolinger, W. A. Higashino, A. Tiwari, M. A. M. Capretz, Data management in cloud environments: NoSQL and NewSQL data stores, Journal of Cloud Computing: Advances, Systems and Applications 2 (1) (2013) 22. doi:10.1186/2192-113X-2-22.

[35] G. C. Deka, A survey of cloud database systems, IT Professional 16 (2) (2014) 50–57.

[36] A. Pavlo, G. Angulo, J. Arulraj, H. Lin, J. Lin, L. Ma, P. Menon, T. C. Mowry, M. Perron, I. Quah, S. Santurkar, A. Tomasic, S. Toor, D. V. Aken, Z. Wang, Y. Wu, R. Xian, T. Zhang, Self-driving database management systems, in: Proceedings of CIDR, 2017.

[37] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, D. Woodford, Spanner: Google's globally distributed database, ACM Transactions on Computer Systems 31 (3) (2013) 1–22. doi:10.1145/2491245.

[38] F. Gessert, W. Wingerath, S. Friedrich, N. Ritter, NoSQL database systems: a survey and decision guidance, Computer Science – Research and Development 32 (3–4) (2017) 353–365. doi:10.1007/s00450-016-0334-3.

[39] T. N. Khasawneh, M. H. AL-Sahlee, A. A. Safia, SQL, NewSQL, and NoSQL databases: A comparative survey, in: 2020 11th International Conference on Information and Communication Systems (ICICS), IEEE, 2020, pp. 013–021.

[40] J. Han, H. E, G. Le, J. Du, Survey on NoSQL database, in: 2011 6th International Conference on Pervasive Computing and Applications (ICPCA), IEEE, 2011, pp. 363–366.

[41] B. G. Tudorica, C. Bucur, A comparison between several NoSQL databases with comments and notes, in: 2011 RoEduNet International Conference 10th Edition: Networking in Education and Research, IEEE, 2011.

[42] A. Bader, O. Kopp, M. Falkenthal, Survey and comparison of open source time series databases, in: Datenbanksysteme für Business, Technologie und Web (BTW 2017) – Workshopband, Lecture Notes in Informatics (LNI), Gesellschaft für Informatik, 2017.

[43] G. Özsoyoğlu, R. T. Snodgrass, Temporal and real-time databases: A survey, IEEE Transactions on Knowledge and Data Engineering 7 (4) (1995) 513–532. doi:10.1109/69.404027.

[44] S. Ramanathan, S. Goel, S. Alagumalai, Comparison of cloud database: Amazon's SimpleDB and Google's Bigtable, in: 2011 International Conference on Recent Trends in Information Systems (ReTIS), 2011. doi:10.1109/ReTIS.2011.6146861.

[45] J. Lu, I. Holubová, Multi-model databases: A new journey to handle the variety of data, ACM Computing Surveys 52 (3) (2019) 1–38. doi:10.1145/3323214.

[46] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, T. Vassilakis, H. Ahmadi, D. Delorey, S. Min, M. Pasumansky, J. Shute, Dremel: A decade of interactive SQL analysis at web scale, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 3461–3472. doi:10.14778/3415478.3415568.

[47] M. Pohl, D. Staegemann, K. Turowski, Data lakehouse for time series data: A systematic literature review, in: 2024 IEEE International Conference on Big Data (BigData), IEEE, 2024.

[48] S. Salehi, A. Selamat, H. Fujita, Systematic mapping study on granular computing, Knowledge-Based Systems 80 (2015) 78–97. doi:10.1016/j.knosys.2015.02.018.

[49] G. Murtaza, L. Shuib, A. W. A. Wahab, G. Mujtaba, H. F. Nweke, M. A. Al-garadi, F. Zulfiqar, G. Raza, N. A. Azmi, Deep learning-based breast cancer classification through medical imaging modalities: State of the art and research challenges, Artificial Intelligence Review 53 (3) (2020) 1655–1720. doi:10.1007/s10462-019-09716-7.

[50] A. Qazi, H. Fayaz, A. Wadi, R. G. Raj, N. Rahim, W. A. Khan, The artificial neural network for solar radiation prediction and designing solar systems: A systematic literature review, Journal of Cleaner Production 104 (2015) 1–12. doi:10.1016/j.jclepro.2015.04.041.

[51] W. Saeed, C. Omlin, Explainable AI (XAI): A systematic meta-survey of current challenges and future opportunities, Knowledge-Based Systems 263 (2023) 110273. doi:10.1016/j.knosys.2023.110273.

[52] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al., The PRISMA 2020 statement: an updated guideline for reporting systematic reviews, BMJ 372 (2021) n71. doi:10.1136/bmj.n71.

[53] T. Dory, B. Mejías, P. Van Roy, N.-L. Tran, Measuring elasticity for cloud databases, in: CLOUD COMPUTING 2011: The Second International Conference on Cloud Computing, GRIDs, and Virtualization, IARIA, 2011.

[54] R. J. Thara, S. Shine, C. Sanu, Optimizing the performance of database as a service (DaaS) model – a distributed approach, in: 2013 Fourth International Conference on Computing, Communications and Networking Technologies (ICCCNT), IEEE, 2013.

[55] L. Ma, D. V. Aken, A. Hefny, G. Mezerhane, A. Pavlo, G. J. Gordon, Query-based workload forecasting for self-driving database management systems, Proceedings of the ACM SIGMOD International Conference on Management of Data (2018) 631–645doi:10.1145/3183713.3196908.

[56] E. Brewer, CAP twelve years later: How the "rules" have changed, Computer 45 (2) (2012) 23–29. doi:10.1109/MC.2012.37.

[57] D. J. Abadi, Consistency tradeoffs in modern distributed database system design: CAP is only part of the story, Computer 45 (2) (2012) 37–42. doi:10.1109/MC.2012.33.

[58] P. Viotti, M. Vukolić, Consistency in non-transactional distributed storage systems, ACM Computing Surveys 49 (1) (2016) 1–34. doi:10.1145/2926965.

[59] T. Rabl, M. Danisch, M. Frank, S. Schindler, H.-A. Jacobsen, Just can't get enough—synthesizing big data benchmarks, IEEE BigData Congress (2019).

[60] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, Proceedings of the ACM SIGMOD International Conference on Management of Data (2021) 1275–1288doi:10.1145/3448016.3452838.

[61] T. Kraska, A. Beutel, E. H. Chi, J. Dean, N. Polyzotis, The case for learned index structures, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2018, pp. 489–504. doi:10.1145/3183713.3196909.

[62] J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, L. Liu, M. Ran, Z. Li, An end-to-end automatic cloud database tuning system using deep reinforcement learning, Proceedings of the ACM SIGMOD International Conference on Management of Data (2019) 415–432doi:10.1145/3299869.3300085.

[63] T. Taipalus, Database management system performance comparisons: A systematic literature review, Journal of Systems and Software 208 (2024) 111872. doi:10.1016/j.jss.2023.111872.

[64] M. T. Samarta, A. A. S. Gunawan, M. E. Syahputra, Systematic literature review and comparative performance analysis of SQL and NoSQL databases in big data applications, in: 2024 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS), IEEE, 2024.

[65] D. M. P. D. Daundasekara, et al., Evaluating performance and user perceptions of multi-cloud database-as-a-service solutions: A mixed-methods study, in: 2025 International Conference on Computing, 2025.

[66] B. Furht, A. Escalante, Handbook of cloud computing, Springer (2010). doi:10.1007/978-1-4419-6524-0.

[67] Z. S. Zubi, On distributed database security aspects, in: 2009 International Conference on Multimedia Computing and Systems (ICMCS), IEEE, 2009.

[68] V. Waghmare, K. Gojre, A. Watpade, Approach to enhancing concurrent and self-reliant access to cloud database: A review, in: 2015 International Conference on Computational Intelligence and Communication Networks (CICN), IEEE, 2015. doi:10.1109/CICN.2015.158.

[69] B. Alouffi, M. Hasnain, A. Alharbi, W. Alosaimi, H. Alyami, M. Ayaz, A systematic literature review on cloud computing security: Threats and mitigation strategies, IEEE Access 9 (2021) 57792–57807. doi:10.1109/ACCESS.2021.3073203.

[70] E. Ferrari, Database as a service: Challenges and solutions for privacy and security, in: 2009 IEEE Asia-Pacific Services Computing Conference (APSCC), IEEE, 2009, pp. 46–51.

[71] K. Munir, Security model for cloud database as a service (DBaaS), in: 2015 International Conference on Cloud Technologies and Applications (CloudTech), IEEE, 2015.

[72] F. A. Khan, M. Jamjoom, A. Ahmad, M. Asif, An analytic study of architecture, security, privacy, query processing, and performance evaluation of database-as-a-service, Transactions on Emerging Telecommunications Technologies 30 (12) (2019) e3814. doi:10.1002/ett.3814.

[73] A. Acar, H. Aksu, A. S. Uluagac, M. Conti, A survey on homomorphic encryption schemes: Theory and implementation, ACM Computing Surveys 51 (4) (2018) 1–35. doi:10.1145/3214303.

[74] R. R. Ravan, N. B. Idris, Z. Mehrabani, A survey on querying encrypted data for database as a service, in: 2013 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC), IEEE, 2013. doi:10.1109/CyberC.2013.12.

[75] L. Ferretti, M. Colajanni, M. Marchetti, Distributed, concurrent, and independent access to encrypted cloud databases, IEEE Transactions on Parallel and Distributed Systems 25 (2) (2014) 437–446. doi:10.1109/TPDS.2013.154.

[76] A. Cuzzocrea, C. Mastroianni, G. M. Grasso, Private databases on the cloud: Models, issues and research perspectives, in: 2016 IEEE International Conference on Big Data (Big Data), 2016, pp. 3656–3661. doi:10.1109/BigData.2016.7841032.

[77] M. F. Gholami, F. Daneshgar, G. Low, G. Beydoun, Cloud migration process—a survey, evaluation framework, and open challenges, Journal of Systems and Software 120 (2016) 31–69. doi:10.1016/j.jss.2016.06.068.

[78] V. Narasayya, S. Das, M. Syamala, B. Chandramouli, S. Chaudhuri, SQLVM: Performance isolation in multi-tenant relational database-as-a-service, Proceedings of CIDR (2015).

[79] S. Jain, D. Moritz, B. Howe, High variety cloud databases, in: 2016 IEEE 32nd International Conference on Data Engineering Workshops (ICDEW), 2016, pp. 12–19. doi:10.1109/ICDEW.2016.7495609.

[80] E. Baralis, A. D. Valle, P. Garza, C. Rossi, F. Scullino, SQL versus NoSQL databases for geospatial applications, in: 2017 IEEE International Conference on Big Data (Big Data), IEEE, 2017, pp. 3388–3397.

[81] K. Mahmood, K. Orsborn, T. Risch, Comparison of NoSQL datastores for large scale data stream log analytics, in: 2019 IEEE International Conference on Smart Computing (SMARTCOMP), IEEE, 2019. doi:10.1109/SMARTCOMP.2019.00093.

[82] H. Tu, Cassandra vs. MongoDB: A systematic review of two NoSQL data stores in their industry uses, in: 2024 IEEE 7th International Conference on Big Data and Artificial Intelligence (BDAI), IEEE, 2024.

[83] W. Hendricks, Review of NoSQL data stores: Using a reactive three-tier application for software developers to achieve a high availability application design architecture, in: 2019 International Conference (IEEE), IEEE, 2019.

[84] N. Tripathi, NoSQL database education: A review of models, tools and teaching methods, Journal of Systems and Software 226 (2025) 112391. doi:10.1016/j.jss.2025.112391.

[85] M. Sheng, S. Wang, Y. Zhang, K. Wang, J. Wang, Y. Luo, R. Hao, MQRLD: A multimodal data retrieval platform with query-aware feature representation and learned index based on data lake, Information Processing & Management 62 (5) (2025) 104101. doi:10.1016/j.ipm.2025.104101.

[86] C. Pahl, P. Jamshidi, O. Zimmermann, Architectural principles for cloud software, ACM Transactions on Internet Technology 18 (2) (2018) 1–23. doi:10.1145/3104028.

[87] T. A. M. Phan, J. K. Nurminen, M. Di Francesco, Cloud databases for internet-of-things data, in: 2014 IEEE International Conference on Internet of Things (iThings), 2014, pp. 117–124. doi:10.1109/iThings.2014.26.

[88] X. Zhao, K.-Y. Lam, Density based learned spatial index for clustered data, Information Systems 135 (2026) 102606. doi:10.1016/j.is.2025.102606.

[89] Z. Sun, X. Zhou, G. Li, Learned index: A comprehensive experimental evaluation, Proceedings of the VLDB Endowment 16 (8) (2023) 1992–2004. doi:10.14778/3594512.3594528.

[90] J. Tan, T. Zhang, F. Li, J. Chen, Q. Zheng, P. Zhang, H. Qiao, Y. Shi, W. Cao, R. Zhang, iBTune: Individualized buffer tuning for large-scale cloud databases, in: Proceedings of the VLDB Endowment, Vol. 12, 2019, pp. 1221–1234. doi:10.14778/3339490.3339503.

[91] B. Hilprecht, C. Binnig, U. Röhm, Learning a partitioning advisor for cloud databases, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 2020, pp. 143–157. doi:10.1145/3318464.3389704.

[92] E. Dritsas, M. Trigka, Machine learning in modern database systems: Techniques, architectures, and deployment challenges, Engineering Applications of Artificial Intelligence 170 (2026) 114225. doi:10.1016/j.engappai.2026.114225.

[93] International Energy Agency, Data centres and data transmission networks, https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks, accessed: 2025-01-15 (2024).

[94] M. Mouhibha, A. Mabrouk, Nosql data warehouse optimizing models: A comparative study of column-oriented approaches, Big Data Research 40 (2025) 100523. doi:10.1016/j.bdr.2025.100523.

[95] L. M. Ferreira, et al., Multidimensional modelling in NoSQL database: A systematic review, in: 2023 18th Iberian Conference on Information Systems and Technologies (CISTI), IEEE, 2023.

[96] M. Mouhibha, A. Mabrouk, An empirically-driven clustering framework for NoSQL data warehouse conversion: Optimizing column family design from relational big data using HDBSCAN, Information and Software Technology 195 (2026) 108117. doi:10.1016/j.infsof.2026.108117.

[97] M. Ma, Z. Yin, S. Zhang, S. Wang, C. Zheng, X. Jiang, H. Hu, C. Luo, Y. Li, N. Qiu, F. Li, C. Chen, D. Pei, Diagnosing root causes of intermittent slow queries in cloud databases, in: Proceedings of the VLDB Endowment, Vol. 13, 2020, pp. 1176–1189. doi:10.14778/3389133.3389136.

[98] K. Affolter, K. Stockinger, A. Bernstein, A comparative survey of recent natural language interfaces for databases, The VLDB Journal 28 (5) (2019) 793–819. doi:10.1007/s00778-019-00567-8.

[99] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction, Advances in Neural Information Processing Systems 36 (2023).

[100] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, Proceedings of the VLDB Endowment 17 (5) (2024) 1132–1145. doi:10.14778/3641204.3641221.

[101] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, Z. Li, MAC-SQL: A multi-agent collaborative framework for text-to-SQL, arXiv preprint arXiv:2312.11242 (2024).

[102] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-labeled dataset for complex and cross-database semantic parsing and text-to-SQL task, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018, pp. 3911–3921. doi:10.18653/v1/D18-1425.

[103] X. Zhou, G. Li, Z. Sun, Z. Liu, W. Chen, J. Wu, J. Liu, R. Feng, G. Zeng, D-bot: Database diagnosis system using large language models, in: Proceedings of the VLDB Endowment, Vol. 17, 2024, pp. 2514–2527. doi:10.14778/3675034.3675043.

[104] S. Xue, C. Jiang, W. Shi, F. Cheng, K. Chen, H. Yang, Z. Zhang, J. He, H. Zhang, G. Wei, W. Zhao, F. Zhou, D. Qi, H. Yi, S. Liu, F. Chen, DB-GPT: Empowering database interactions with private large language models, arXiv preprint arXiv:2312.17449 (2024).

[105] J. Lao, Y. Wang, Y. Li, J. Wang, Y. Zhang, Z. Cheng, W. Chen, M. Tang, J. Wang, GPTuner: A manual-reading database tuning system via GPT-guided bayesian optimization, Proceedings of the VLDB Endowment 17 (8) (2024) 1939–1952. doi:10.14778/3659437.3659449.

[106] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv preprint arXiv:2312.10997 (2024).

[107] H. J. Bhatti, B. B. Rad, Databases in cloud computing: A literature review, International Journal of Information Technology and Computer Science 9 (4) (2017) 9–17. doi:10.5815/ijitcs.2017.04.02.